

UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
CENTRO DE ENGENHARIA ELÉTRICA E INFORMÁTICA
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

VITOR LUCAS SOUZA DE HOLANDA

PLATAFORMA DE PROCTORING MULTIMODAL PARA SUPORTE À DECISÃO

CAMPINA GRANDE - PB
2026

VITOR LUCAS SOUZA DE HOLANDA

PLATAFORMA DE PROCTORING MULTIMODAL PARA SUPORTE À DECISÃO

Trabalho de Conclusão de Curso apresentado ao Curso Bacharelado em Ciência da Computação do Centro de Engenharia Elétrica e Informática da Universidade Federal de Campina Grande, como requisito parcial para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Thiago Emmanuel Pereira da Cunha Silva

CAMPINA GRANDE - PB
2026

VITOR LUCAS SOUZA DE HOLANDA

PLATAFORMA DE PROCTORING MULTIMODAL PARA SUPORTE À DECISÃO

Trabalho de Conclusão de Curso apresentado ao Curso Bacharelado em Ciência da Computação do Centro de Engenharia Elétrica e Informática da Universidade Federal de Campina Grande, como requisito parcial para obtenção do título de Bacharel em Ciência da Computação.

BANCA EXAMINADORA:

Thiago Emmanuel Pereira da Cunha Silva
Orientador - UASC/CEEI/UFCG

Examinador 1
UASC/CEEI/UFCG

Examinador 2
UASC/CEEI/UFCG

Trabalho aprovado em: _____ de _____ de 2026.

Plataforma de Proctoring Multimodal para Suporte à Decisão

Vitor Lucas Souza de Holanda
vitor.holanda@ccc.ufcg.edu.br
Universidade Federal de Campina Grande
Campina Grande, Paraíba, Brasil

Thiago Emmanuel Pereira da Cunha Silva
temmanuel@computacao.ufcg.edu.br
Universidade Federal de Campina Grande
Unidade Acadêmica de Sistemas e Computação (UASC) – CEEI
Campina Grande, Paraíba, Brasil

Resumo

A realização de provas de conhecimento em ambientes remotos impõe o desafio de validar a autoria das soluções sem comprometer a experiência do usuário. Este trabalho propõe uma plataforma de *proctoring* de código aberto projetada não como um juiz automatizado, mas como uma ferramenta de auditoria e suporte à decisão docente. A arquitetura destaca-se pela utilização de múltiplos módulos de captura e análise, processando sinais de áudio, vídeo da *webcam*, dinâmica de digitação e gravação integral do *desktop* para reprodução da sessão. A solução utiliza um módulo de execução de código Python baseado em Pyodide, permitindo o processamento diretamente no navegador, além de realizar análise de similaridade textual e normalizada para detecção de plágio. A plataforma coleta sinais multimodais para gerar relatórios individuais, permitindo que o avaliador humano discirna com precisão entre comportamentos naturais e potenciais violações de integridade. O resultado é um painel de auditoria consolidado que apresenta anomalias de digitação, métricas de similaridade lógica, estimativa de pose da cabeça em cinco direções e identificação de múltiplos falantes no ambiente de prova.

Palavras-chave

Proctoring digital; Suporte à decisão; Pyodide; Multimodalidade; Integridade acadêmica.

1 Introdução

A execução de avaliações de conhecimento e testes de aptidão é fundamental tanto para a mensuração do aprendizado acadêmico quanto para processos de e recrutamento profissional [1]. Contudo, a transição para modelos remotos impõe a necessidade de mecanismos robustos para a verificação da conduta do candidato e a validação da autoria das soluções produzidas, coletando dados que permitam inferir a legitimidade do código submetido.

Atualmente, diversas plataformas de avaliação remotas limitam-se à captura passiva de tela ou câmera, enquanto soluções proprietárias consolidadas no mercado frequentemente geram relatórios opacos sobre suspeitas de fraude, carecendo de transparência quanto às evidências que sustentam tais alertas [3, 4]. Em contrapartida, sistemas educacionais de código aberto amplamente utilizados em cursos de graduação como os Ambientes Virtuais de Aprendizagem (AVA) carecem de infraestruturas integradas para monitorar a integridade das atividades em tempo real. A ausência desses mecanismos torna crítica, visto que a prática de plágio em estágios iniciais de aprendizagem compromete o desenvolvimento de competências técnicas essenciais ao longo da formação acadêmica [2, 5].

Neste contexto, este trabalho apresenta o **ProctorLCC**, uma plataforma de código aberto equipada com módulos de análise multimodal. A viabilidade de uma solução de *proctoring* desta natureza repousa sobre o equilíbrio entre técnicas de visão computacional, processamento de sinais e segurança da informação. Ao integrar sinais de áudio, vídeo, dinâmica de digitação e análise de similaridade de código, o ProctorLCC fundamenta seus pilares tecnológicos na transparência algorítmica. O objetivo central é fornecer ao avaliador um painel de indícios forenses, garantindo que a decisão final sobre a validade da prova seja humana e fundamentada em dados técnicos robustos [6].

Este documento está estruturado da seguinte forma. A Seção 2 apresenta a fundamentação teórica que sustenta o trabalho. A Seção 3 contextualiza a pesquisa em relação a outros trabalhos da área. A Seção 4 descreve a metodologia de desenvolvimento e os materiais utilizados. Os resultados experimentais são apresentados na Seção 5 e analisados criticamente na Seção 6. Por fim, a Seção 7 aponta direções para trabalhos futuros e a Seção 8 oferece as conclusões gerais deste trabalho. Todo o código desenvolvido está disponível em: https://github.com/VitorHolandaI/tcc e a aplicação acessível em https://prova.bitssand.blog.

2 Fundamentação Teórica

A viabilidade de uma plataforma de *proctoring* multimodal repousa sobre o equilíbrio entre técnicas de visão computacional, processamento de sinais e segurança da informação. Esta seção fundamenta os pilares tecnológicos do ProctorLCC.

2.1 Integridade e Vigilância Digital

A manutenção da integridade em exames remotos fundamenta-se na criação de ambientes computacionais restritos, amplamente conhecidos como modo quiosque (Kiosk Mode), que isolam o processo de avaliação e bloqueiam aplicações não autorizadas [7]. O sistema proposto estabelece essa vigilância condicionando o início da prova à concessão obrigatória de permissões de hardware (microfone e câmera) e ao compartilhamento da tela inteira do sistema operacional, centralizando o controle de conformidade diretamente na interface do usuário.

Diferente de abordagens permissivas que permitem o compartilhamento de apenas uma aba do navegador, a exigência da captura total da tela, fundamentada na *Screen Capture API*, visa mitigar o uso de recursos externos, como ferramentas de comunicação paralela ou janelas ocultas de navegadores. Complementarmente, a aplicação utiliza a *Fullscreen API* para garantir a imersão total e a *Page Visibility API* para monitorar eventos de perda de foco. Conforme discutido por Dendir e Maxwell [5], a imposição dessas restrições

técnicas atua como um fator de dissuasão primário, reduzindo significativamente a carga cognitiva do avaliador ao filtrar tentativas triviais de violação de integridade.

2.2 Diarização de Locutores e Modelos Pré-treinados

A Diarização de Locutores é a técnica de processamento de sinais voltada a resolver o problema de segmentar um fluxo de áudio e identificar "quem falou quando". Teoricamente, esse processo baseia-se na extração de características acústicas que distinguem a anatomia vocal de diferentes indivíduos, independentemente do idioma ou conteúdo falado.

Para operacionalizar essa análise, o sistema utiliza modelos de redes neurais convolucionais pré-treinados. Esses modelos são desenvolvidos através de aprendizado profundo (*Deep Learning*) em vastas bases de dados de fala humana, com o objetivo de aprender a mapear ondas sonoras em um espaço vetorial de alta dimensionalidade (*embeddings*). A finalidade desse modelo no contexto de *proctoring* é permitir que o sistema diferencie a voz do candidato de vozes externas de forma automatizada, fornecendo indícios de auxílio não autorizado sem a necessidade de um treinamento prévio para cada novo ambiente de prova.

A eficácia do módulo reside na geração de métricas derivadas da linha do tempo (*speaker timeline*): a detecção de múltiplos locutores (*overlapping speakers*), a contabilização de falantes únicos e o percentual de silêncio. No ProctorLCC, intervalos de sobreposição de vozes inferiores a 0,5 segundos entre locutores diferentes são classificados como indícios de interação externa, fornecendo ao avaliador um mapa exato de eventos anômalos para auditoria.

2.3 Estimativa de Pose de Cabeça e Geometria Projetiva

A análise visual da atenção do discente fundamenta-se na Estimativa de Pose de Cabeça, uma técnica de visão computacional voltada para determinar a orientação facial em relação à câmera. O sistema utiliza o *framework* **MediaPipe** [8], empregando o modelo *Face Mesh* para extrair 468 marcos faciais tridimensionais com baixo custo computacional. Para garantir a estabilidade do rastreamento, são selecionados seis pontos anatômicos estratégicos: a ponta do nariz, os cantos internos dos olhos, as extremidades da boca e o mento.

A implementação técnica resolve o problema de *Perspective-n-Point* (PnP) utilizando a biblioteca **OpenCV**, mapeando os pontos 2D da imagem para um modelo 3D antropométrico genérico. O vetor de rotação resultante é processado via método de Rodrigues, implementada na função Rodrigues() da biblioteca OpenCV [9] e decomposto nos ângulos de Euler: $Pitch$ (inclinação vertical), Yaw (rotação horizontal) e *Roll* (inclinação lateral).

No ambiente de prova, esses dados são convertidos em métricas de suporte à decisão, classificando o olhar em categorias discretas como frente, esquerda, direita, cima e baixo. Conforme a lógica de auditoria implementada, o sistema identifica automaticamente a ausência de face ou períodos de desvio de atenção superiores a dez segundos, gerando um rastro forense robusto para a avaliação docente.

2.4 Análise de Similaridade de Código e Normalização Estrutural de Código

A detecção de plágio em código-fonte transcende a simples comparação textual, uma vez que algoritmos podem ser representados por diferentes grafias mantendo a mesma lógica. O ProctorLCC fundamenta essa análise no algoritmo de Ratcliff e Obershelp [10], que calcula o grau de similaridade entre duas sequências através da busca pela maior substring comum e o processamento recursivo das substrings adjacentes.

Contudo, para mitigar técnicas de camuflagem como a renomeação de variáveis ou alteração de literais o sistema aplica uma etapa de **Normalização Estrutural** antes da comparação. Teoricamente, este processo reduz a "entropia superficial"do código ao:

(1) **Tokenizar e Filtrar:** Remover elementos sem valor semântico para a lógica, como comentários e espaços em branco excessivos.
(2) **Canalização de Literais:** Substituir *strings* e valores numéricos por marcadores genéricos (ex: `"STR"` e `"NUM"`).

Essa abordagem permite que o cálculo de similaridade foque na topologia do algoritmo. Ao comparar o texto bruto em paralelo com a versão normalizada, o sistema gera métricas que auxiliam o avaliador a identificar casos onde a estrutura lógica foi preservada, mesmo que a representação textual tenha sido alterada pelo aluno.

2.5 Biometria Comportamental e Detecção de Anomalias

Diferente das biometrias fisiológicas, a dinâmica de digitação (*keystroke dynamics*) analisa os padrões rítmicos de interação com o teclado para formar uma "assinatura de digitação"[11]. No escopo desta plataforma, a técnica é aplicada para a **detecção de anomalias comportamentais**, caracterizando-se como uma evolução da infraestrutura experimental proposta por Santos Filho [11]. O objetivo é discernir entre o padrão de digitação orgânico e comportamentos suspeitos, como a inserção artificial de blocos de código ou ritmos incompatíveis com a escrita humana.

A extração de características baseia-se em dois descritores temporais fundamentais validados nos cenários de teste de Santos Filho [11]:

- **Hold Time (HT):** A duração do pressionamento de uma tecla, refletindo aspectos motores do usuário.
- **Flight Time (FT):** O intervalo entre pressões de teclas consecutivas, frequentemente associado à carga cognitiva e familiaridade com o código.

Para quantificar o grau de suspeição, o ProctorLCC processa esses descritores através de duas variáveis estatísticas fundamentadas na metodologia de Santos Filho [11]:

2.5.1 Distância de Manhattan para Desvio de Padrão. A métrica utilizada para medir a disparidade entre o comportamento de um segmento e a média global é a **Distância de Manhattan** (L_1 *norm*). Ela calcula a soma das diferenças absolutas entre os vetores de métricas temporais. Formalmente, para dois vetores P (padrão médio) e Q (segmento atual), a distância d é dada por:

$$d(\mathbf{P}, \mathbf{Q}) = \sum_{i=1}^{n} |P_i - Q_i|$$

Conforme observado em experimentos anteriores, distâncias elevadas evidenciam desvios substanciais compatíveis com comportamento artificial.

2.5.2 Detecção de Outliers via Z-Score. Complementarmente, utiliza-se o **pontuação Padrão** (Z) para identificar *outliers*. Teclas com tempos que divergem da média em um fator superior a 3.0 são marcadas como anomalias. O cálculo do Z-score é definido por:

$$z = \frac{x - \mu}{\sigma}$$

Onde μ é a média e σ o desvio padrão. Valores elevados indicam pausas anormais ou comportamentos não humanos.

A integração dessas métricas, agindo como variáveis de suporte à decisão, permite que a plataforma gere um rastro forense robusto ao correlacionar a variabilidade rítmica com comandos de sistema (copiar e colar).

2.6 Arquitetura de Microsserviços

A arquitetura de microsserviços consiste na decomposição de uma aplicação em um conjunto de serviços modulares e independentes, onde cada componente executa seu próprio processo. Conforme postulado por Lewis e Fowler [12], essa abordagem substitui a estrutura monolítica por uma orquestração de processos leves que se comunicam via rede. Essa escolha estrutural fundamenta-se em três pilares centrais:

- **Flexibilidade:** Capacidade do sistema em se adaptar a mudanças tecnológicas e suportar modificações em serviços específicos sem impactar a aplicação global.
- **Modularidade:** Organização em componentes isolados que contribuem para o comportamento geral, permitindo que falhas sejam confinadas a um único módulo e facilitando a manutenção individual.
- **Evolução:** Estrutura que permite a integração contínua de novos recursos e algoritmos de detecção de anomalias enquanto o sistema permanece disponível e operacional.

No contexto desta plataforma de proctoring, a arquitetura de microsserviços é fundamental para isolar os módulos de análise computacionalmente intensivos como o processamento de vídeo e a diarização de áudio da API principal que gerencia as provas. Essa separação garante que a interface do usuário permaneça responsiva e estável, mesmo durante picos de processamento, e permite que novos módulos de detecção (como uma futura análise de dispositivos móveis) sejam desenvolvidos e implantados de forma independente, sem risco para a operação do sistema como um todo.

3 Trabalhos Relacionados

O ecossistema atual de avaliação técnica remota apresenta uma dicotomia clara entre plataformas comerciais de alto custo, dotadas de inteligência artificial generativa para detecção de fraude, e soluções de código aberto que, embora acessíveis, operam de forma fragmentada. A Tabela 1 sintetiza as principais diferenças funcionais e econômicas entre as soluções existentes e a arquitetura proposta neste trabalho.

No segmento comercial, ferramentas como HackerRank[13] e WeCP (Sherlock) [14] definem o estado da arte em 2025. O HackerRank, por exemplo, implementou recentemente um motor de detecção de plágio com 93% de precisão que analisa a dinâmica de digitação (*keystroke dynamics*) para identificar se o código foi gerado por IA ou copiado externamente, correlacionando a velocidade de escrita com a complexidade algorítmica. Da mesma forma, o WeCP utiliza agentes de IA (Sherlock) para monitorar o ambiente físico via webcam, detectando múltiplos rostos ou o uso de dispositivos secundários em tempo real.

Contudo, essas tecnologias impõem barreiras financeiras significativas: o plano inicial do HackerRank custa aproximadamente US$ 165 mensais (faturado anualmente), enquanto o WeCP inicia em faixas próximas a US$ 240 mensais. Alternativas emergentes como o Utkrusht[15] tentam mitigar isso com modelos de pagamento por uso focados em simulações de trabalho real, mas ainda dependem de código proprietário fechado.

No cenário de código aberto, a fragmentação é o principal desafio. O Judge0 [17] consolidou-se como um padrão robusto para a execução de código em **ambiente isolado** via API, suportando mais de 60 linguagens. No entanto, o Judge0 é estritamente um motor de execução; ele não possui interface de prova nem mecanismos de vigilância. Para garantir a integridade do exame, instituições frequentemente o combinam com o Safe Exam Browser (SEB) [16]. Embora o SEB seja eficaz em bloquear o sistema operacional em **modo quiosque** impedindo a navegação não autorizada, ele falha em monitorar a identidade do aluno, não possuindo recursos nativos de gravação de vídeo ou análise biométrica.

Este trabalho propõe preencher essa lacuna ao integrar, em uma única plataforma de código aberto, a capacidade de execução de código (similar ao Judge0) com uma camada de vigilância multimodal. Diferente de soluções comerciais focadas na interrupção punitiva, a arquitetura aqui desenvolvida prioriza a coleta de evidências forenses vídeo, áudio e dinâmica de digitação para auditoria posterior, oferecendo uma alternativa gratuita e pedagogicamente orientada para instituições de ensino que não podem arcar com licenciamentos corporativos.

4 Metodologia

A construção da plataforma seguiu um processo de desenvolvimento iterativo, inspirado em metodologias ágeis, que pode ser decomposto nas seguintes etapas de engenharia de software.

4.1 Levantamento e Análise de Requisitos

O levantamento de requisitos foi realizado por meio de entrevistas semiestruturadas com potenciais usuários e especialistas no domínio de avaliação acadêmica. A principal entrevista foi conduzida com o professor Thiago Emmanuel, orientador deste trabalho e pesquisador na área, que atuou como o principal cliente (*stakeholder*) do projeto.

A partir dessas conversas, os requisitos foram modelados como Histórias de Usuário (*User Stories*) para manter o foco na perspectiva do usuário. Os principais requisitos funcionais (RF) e não funcionais (RNF) identificados foram:

- **[RF01 - Visão do Docente]:** Como professor, eu quero um painel de auditoria que consolide os dados de vídeo, áudio e digitação de um aluno, para que eu possa tomar uma decisão informada sobre a integridade da prova.

Tabela 1: Comparativo de Soluções de Proctoring e Avaliação (2025/2026)

Plataforma	Custo (Est.)	Webcam	Key.*	Diferenciais e Detalhes
ProctorLCC	**Código Aberto (Open Source)**	✓	✓	**Única Open Source com biometria multimodal (Vídeo + Dinâmica de Digitação) e execução local.**
Judge0	Grátis**	×	×	Motor de execução robusto (60+ linguagens), mas sem interface de prova ou vigilância [17, 18].
Safe Exam Browser	Código Aberto (Open Source)	×	×	Bloqueia o SO em **modo quiosque**; padrão de mercado, mas não grava vídeo nem detecta IA [19].
HackerRank	$165+/mês	✓	✓	Líder de mercado; IA detecta plágio e anomalias de digitação (93% precisão) [21].
WeCP (Sherlock)	$240/mês	✓	✓	Agente 'Sherlock' usa IA para detectar objetos, múltiplos rostos e desvio de olhar [14].
TestTrick	$35/mês	✓	×	Alternativa de baixo custo; inclui monitoramento básico de tela e webcam [23].
Codility	$500/mês	✓	✓	Foco em compliance e anti-vazamento de questões para grandes empresas [24].
Quilgo	$160/mês	✓	✓	Plugin focado em escolas; cobra por assentos simultâneos (ex: 500 alunos) [25].

Legenda: C.A. = Código Aberto. *Key. = Keystroke Dynamics (Dinâmica de Digitação). **Judge0 é grátis para auto-hospedagem.
Fontes: Elaborado pelo autor (2026) com base em [17, 19, 21–25].

- **[RF02 - Visão do Aluno]:** Como aluno, eu quero um ambiente de prova estável e que não interfira no meu desempenho, para que eu possa focar na resolução das questões.
- **[RNF01 - Desempenho]:** O sistema de monitoramento não deve sobrecarregar o computador do aluno, mantendo a interface da prova responsiva.
- **[RNF02 - Escalabilidade e Manutenibilidade]:** A arquitetura deve permitir que novos módulos de análise sejam desenvolvidos e implantados de forma independente, sem a necessidade de reescrever o sistema inteiro.

4.2 Desenho da Arquitetura e Implementação

Com base nos requisitos, especialmente o RNF02, optou-se por uma **arquitetura de microsserviços**. Essa decisão, detalhada na Seção 2, foi fundamental para isolar as tarefas de processamento intensivo (análise de mídia) da API principal, garantindo o cumprimento do RNF01.

O desenvolvimento foi realizado de forma iterativa, com a implementação e validação de cada microsserviço de forma independente, culminando na integração final da plataforma descrita nas subseções a seguir.

4.3 Arquitetura e Comunicação entre Serviços

O sistema, denominado *ProctorLCC*, é estruturado em serviços independentes que operam de forma coordenada para capturar e analisar dados em tempo real. A comunicação entre a interface do usuário e os serviços de retaguarda ocorre via APIs REST e protocolos *WebSocket* para transmissões de baixa latência.

4.3.1 Barramento de Captura e Streaming. A plataforma implementa serviços especializados para o gerenciamento de fluxos de mídia:

- **Recording, Webcam & Audio Services:** Utilizam *WebSockets* para o recebimento de segmentos binários de vídeo e áudio provenientes do *front-end*. A captura é realizada através da *MediaStream Recording API*, que gera *containers* **WebM** com codificação de áudio **Opus** (48kHz). No servidor, esses fluxos são processados via **FFmpeg** para garantir a integridade dos arquivos e a sincronização precisa entre as trilhas de tela e câmera.
- **Persistência de Mídia:** Os arquivos são segmentados de forma incremental durante a execução da prova. Essa abordagem permite a recuperação de dados em caso de falha de conexão ou interrupções inesperadas, sendo a concatenação final dos fragmentos realizada apenas após a submissão definitiva pelo usuário.

4.3.2 Módulos de Análise Comportamental. Os serviços de análise processam os dados de forma assíncrona para não onerar a API principal:

- **Serviço de Análise Visual (Gaze Tracking):** Implementa o *MediaPipe Face Mesh* para estimativa de pose da cabeça em 3D. O sistema calcula o desvio angular em relação ao monitor, gerando métricas de persistência do olhar em direções suspeitas.
- **Keystroke Service:** Analisa a biometria comportamental através da dinâmica de digitação, monitorando métricas de *Hold Time* e *Flight Time*. Aplica-se a análise de *Z-score* para detectar comportamentos anômalos e o uso de comandos de sistema como colagens frequentes.
- **Similarity Service:** Executa a detecção de plágio comparando as submissões. O algoritmo realiza uma normalização estrutural (removendo comentários e espaços) antes da comparação textual, permitindo identificar lógicas idênticas mesmo com alteração de variáveis.

4.4 Implementação do Frontend e Fluxo de Operação

O *frontend* da plataforma foi desenvolvido utilizando **React 19** e **TypeScript**, estruturado para guiar o usuário desde a criação da avaliação até a auditoria dos resultados.

4.4.1 Acesso e Gestão de Perfis. O ponto de entrada do sistema é a autenticação via Google OAuth. Após a identificação, o sistema renderiza o painel correspondente ao papel do usuário. O professor visualiza o gerenciamento global de provas, enquanto o aluno visualiza as avaliações autorizadas, figura 3.

4.4.2 Dashboard do Professor. Todo professor possui, em seu dashboard, as provas que estão em andamento (ou seja, dentro do horário de aplicação), as provas já finalizadas, as provas passadas e as provas agendadas. Também estão disponíveis as opções de editar uma prova ou criar uma nova, bem como visualizar a prova em andamento com suas respectivas submissões.

A Figura 4 mostra a primeira parte do painel, enquanto a Figura 5 apresenta exemplos de provas agendadas e provas já realizadas.

4.4.3 Configuração de Exames. Na criação de uma prova, o docente cadastra as questões com suas respectivas descrições, pesos e exemplos, conforme ilustrado na Figura 6 e na figura 7.

4.4.4 Dashboard do Aluno. Todo aluno possui, em seu dashboard, as provas que estão em andamento (ou seja, dentro do horário de realização), as provas já finalizadas, as provas passadas e as provas agendadas, conforme ilustrado na Figura 8.

4.4.5 Segurança de Interface: O Componente ScreenGuard. Antes do acesso ao enunciado, o sistema ativa o **ScreenGuard**. Esse componente atua como um mecanismo de proteção da interface:

- **Validação:** O aluno passa por uma verificação das permissões de mídia e de hardware.
- **Restrição:** O sistema exige o modo de tela cheia (*Fullscreen API*). Tentativas de alternar janelas (por exemplo, *Alt+Tab*) acionam o evento *visibilitychange*, que é registrado no *backend*.

A Figura 9 demonstra o funcionamento desse componente.

4.4.6 Ambiente de Execução e Captura de Sinais. No ambiente de prova, o aluno utiliza o editor **Monaco**. O *hook* customizado `useKeystrokeCapture` intercepta a dinâmica de digitação, enquanto os serviços de transmissão contínua enviam os dados de vídeo e áudio por meio de *WebSockets*.

A Figura 10 ilustra esse ambiente.

4.4.7 Auditoria e Painel de Suporte à Decisão. Durante a realização da prova, ou no período pós-prova, o professor pode acessar a interface de auditoria para visualizar as submissões de cada aluno. Essa página consolida o código submetido com as análises realizadas, permitindo a atribuição de notas fundamentada em evidências.

As Figuras 11 e 12 ilustram essa interface.

4.4.8 Análises de Suporte à Decisão. Após a realização da prova, o professor tem acesso às submissões de cada aluno, podendo executar as análises propostas neste trabalho. Essas análises incluem o processamento de áudio e vídeo da webcam, a avaliação da dinâmica de digitação (*keystroke dynamics*) e o cálculo de similaridade de código entre as submissões.

Além disso, a plataforma permite a reprodução sincronizada das gravações de tela (desktop) e da webcam, armazenadas durante o exame, possibilitando uma análise visual complementar.

A análise visual é ilustrada na Figura 13; a análise de áudio, na Figura 14; a análise da dinâmica de digitação, na Figura 15; a análise de similaridade (plágio), na Figura 16; e a reprodução das gravações, na Figura 17.

Por fim, o aluno tem acesso à sua página de resultados, contendo o *feedback* do docente e sua pontuação final.

4.5 Arquitetura de Backend e Microsserviços Especializados

A retaguarda da plataforma ProctorLCC é fundamentada em uma arquitetura de microsserviços distribuída, composta por módulos independentes que operam em contêineres isolados. Essa abordagem foi adotada para isolar tarefas computacionalmente intensivas, como o processamento de vídeo e a inferência de modelos de IA, da API principal, garantindo a estabilidade e a responsividade do sistema durante picos de submissões simultâneas.

Utiliza-se parcialmente o Modelo C4 para representar a arquitetura em seus diferentes níveis de abstração e, posteriormente, realiza-se um detalhamento mais aprofundado dos serviços que compõem o sistema.

4.5.1 Visão de Contexto (C4 Level 1). O sistema ProctorLCC atua como um mediador entre o discente, que realiza a avaliação, e o docente, que audita os resultados. A autenticação é delegada ao serviço Firebase Auth, que provê o fluxo de login via Google OAuth e emite o JSON Web Token (JWT) necessário para autorizar as requisições em todos os serviços a figura 18 mostra o diagrama.

A Tabela 2 sintetiza os serviços que compõem o ecossistema, destacando suas tecnologias base e responsabilidades primárias.

Tabela 2: Microsserviços do backend e suas principais características

Serviço	Tecnologia Base	Responsabilidade
Main API	FastAPI / PostgreSQL	Orquestração, autenticação e persistência
Recording Service	FastAPI / WS*	Captura contínua de tela
Analysis Service	MediaPipe / OpenCV	Análise de olhar (*gaze*)
Webcam Service	FastAPI / WS*	Captura de vídeo
Audio Service	Pyannote.audio	Diarização de áudio
Keystroke Service	FastAPI	Dinâmica de digitação
Similarity Service	FastAPI	Detecção de plágio

*__Legenda: WS__ refere-se ao protocolo *WebSocket*, utilizado para a transmissão bidirecional e contínua de mídia em tempo real.

4.5.2 Visão de Contêineres (C4 Level 2). A nível de contêineres, o sistema é orquestrado via Docker Compose em oito unidades independentes. O Nginx atua como proxy reverso e gateway, roteando chamadas REST para a Main API e promovendo conexões para o

protocolo WebSocket nos serviços de captura, o diagrama da figura 19 mostra detalhes.

- **Main API (FastAPI):** Orquestrador central que gerencia o CRUD de provas, questões e o banco PostgreSQL.
- **Captura de Mídia (Recording & Webcam):** Serviços que recebem chunks binários WebM via WebSocket e utilizam FFmpeg para persistência resiliente.
- **Módulos de Análise:** Serviços assíncronos para Gaze Tracking (MediaPipe), Diarização de Áudio (Pyannote), Dinâmica de Digitação e Similaridade de Código.

4.5.3 Arquitetura Interna e Fluxo de Captura (C4 Nivel 3). A API principal segue o *Repository Pattern*, promovendo a separação entre a lógica de negócio (Controladores), a camada de persistência de dados (Repositórios) e a validação de dados (Schemas Pydantic). Essa abordagem favorece modularidade, testabilidade e manutenção do código a figura .

Para a captura de sinais, o fluxo operacional é regido por uma coreografia de baixa latência, estruturada da seguinte forma:

- O componente *ScreenGuard*, presente no Frontend, estabelece a conexão via WebSocket com o serviço de captura.
- Os dados de mídia são transmitidos em lotes (*chunks*), estratégia adotada para mitigar perdas decorrentes de instabilidades na rede.
- Ao término da captura, o serviço realiza a concatenação dos segmentos recebidos e notifica a API principal para o disparo das análises assíncronas.

A figura 20 mostra o diagrama.

4.5.4 API Principal e Padrões de Projeto. A API principal (Main API) atua como o orquestrador central do sistema e funciona como o único ponto de entrada exposto ao *frontend*. O banco de dados relacional adotado é o PostgreSQL, cujo acesso é intermediado pelo mapeador objeto-relacional (ORM) SQLAlchemy, utilizando o padrão *Unit of Work* com sessões gerenciadas de forma autônoma por injeção de dependência.

A camada de persistência e as regras de negócio foram estruturadas seguindo o **Repository Pattern**, estabelecendo uma clara separação de responsabilidades em três níveis operacionais:

- **Controladores:** Responsáveis por gerenciar as rotas HTTP, receber as requisições, validar autorizações e delegar a lógica de negócios para os repositórios adequados.
- **Repositórios:** Camada que encapsula toda a lógica de persistência e consultas ao banco de dados, isolando o restante da aplicação das complexidades operacionais do SQL.
- **Esquemas (Schemas):** Modelos de dados responsáveis pela validação, desserialização de entradas e serialização de saídas, garantindo a integridade tipográfica das informações trafegadas.

O modelo de dados centraliza as entidades fundamentais do domínio educacional: `Professor`, `Aluno`, `Prova`, `Questao` e `Submissao`. A entidade `Prova`, por exemplo, gerencia restrições críticas como as linguagens de programação permitidas e a lista de controle de acesso de e-mails autorizados). Paralelamente, a entidade `Submissao` atua como um registro das submissoes, persistindo o código final, o resultado da execução e os dados brutos de interação em formato estruturado (como a dinâmica de digitação), que são posteriormente consumidos pelos serviços de análise.

4.5.5 Mecanismos de Autenticação e Autorização. A segurança do sistema é fundamentada em uma arquitetura *stateless* baseada em *tokens*. A autenticação inicial é delegada ao serviço Firebase, que emite um *JSON Web Token* (JWT) repassado pela interface do usuário e validado criptograficamente pela API principal a cada requisição.

A autorização e o controle de acesso operam sob o modelo de **Controle de Acesso Baseado em Papéis (RBAC)**, cujos papéis são inferidos dinamicamente a partir do domínio de e-mail institucional do usuário autenticado:

- E-mails terminados em `@computacao.ufcg.edu.br` recebem privilégios de **Docente**, com permissões irrestritas para criação, gerenciamento e auditoria de suas respectivas provas.
- E-mails terminados em `@ccc.ufcg.edu.br` são classificados como **Discentes**. Seu acesso é estritamente limitado à resolução de provas, condicionado à validação de presença na lista de acesso da avaliação e ao respeito à janela temporal de execução.

Adicionalmente, políticas de propriedade no nível do repositório garantem que um docente não possa alterar provas de outros professores e que alunos não tenham acesso a rotas administrativas.

4.5.6 Visão Geral dos Microsserviços de Captura e Análise. Para garantir a escalabilidade do monitoramento, os serviços de captura de mídia e processamento de Inteligência Artificial foram totalmente desacoplados da Main API. Uma visão geral destes componentes modulares é apresentada a seguir, sendo seus algoritmos e métricas detalhados de forma aprofundada nas subseções subsequentes:

- **Captura Contínua de Mídia:** Conjunto de microsserviços (*Recording Service* e *Webcam Service*) dedicados à recepção via *WebSockets* e ao armazenamento resiliente (em segmentos delimitados) dos fluxos de vídeo da câmera e da tela do aluno.
- **Análise Comportamental Multimodal:** Módulos assíncronos que processam os dados brutos para extrair métricas de suspeição. Incluem o rastreamento da direção do olhar (*Analysis Service*), a detecção de múltiplas vozes no ambiente de prova (*Audio Service*) e a avaliação rítmica motora de escrita (*Keystroke Service*).
- **Auditoria e Integridade em Tempo Real:** Serviços voltados à garantia de autoria, englobando o cálculo de similaridade estrutural de código (*Similarity Service*) e mecanismos de invalidação automática caso o discente exceda o limite permitido de evasões da interface de prova.

4.5.7 Serviços de Captura e Transmissão Contínua. O monitoramento visual da prova apoia-se em serviços dedicados para a gravação da tela do ambiente operacional e da câmera frontal do usuário. Ambos os serviços operam de forma assíncrona, utilizando o protocolo *WebSocket* para recepcionar fluxos binários contínuos diretamente do navegador do aluno. Para assegurar o controle de acesso, o servidor intercepta a conexão inicial exigindo a transmissão de um *JSON Web Token* (JWT). Este token é validado localmente pelo Firebase Admin SDK e o e-mail extraído é confrontado com a lista de permissões (*whitelist*) vinculada à avaliação em curso.

Visando mitigar a perda de dados em cenários de instabilidade na conexão de rede do discente, o serviço de gravação de tela implementa uma estratégia de particionamento físico da mídia. Os dados recebidos são escritos de forma sequencial em segmentos temporários limitados a 50 MB. Ao término da sessão, o servidor aciona a ferramenta FFmpeg para realizar a concatenação binária por cópia de fluxo, unificando os segmentos em um único contêiner WebM íntegro. A adoção deste formato baseia-se na utilização do codec de vídeo **VP8**, nativamente suportado pela *MediaRecorder API* dos navegadores. O VP8 oferece um excelente balanço entre taxa de compressão e preservação de detalhes textuais, exigindo baixo processamento do lado do cliente para a codificação da tela em tempo real. Essa arquitetura modular permite que, em caso de quedas de conexão, o aluno retome a prova sem corromper o vídeo já gravado, uma vez que a nova sessão gerará um novo segmento a ser anexado posteriormente.

O serviço de captura da webcam opera sob o mesmo paradigma de comunicação e encapsulamento (WebM). Para a captura da trilha sonora do ambiente, a plataforma adota o codec de áudio **Opus**. A escolha deste padrão é justificada por sua alta eficiência algorítmica na compressão de voz e excepcional baixa latência, características essenciais para viabilizar a transmissão contínua de áudio sem sobrecarregar a largura de banda da rede do candidato. O artefato audiovisual completo gerado por este serviço atua, posteriormente, como entrada direta para os módulos assíncronos de análise visual e diarização de locutores após a conclusão do exame.

4.5.8 Análise de Áudio. O módulo de análise de áudio é responsável por identificar a presença de múltiplos locutores durante a realização da prova, servindo como um dos indicadores de possível fraude acadêmica. O áudio é capturado em tempo real por meio de uma conexão WebSocket estabelecida entre o navegador do aluno e o servidor, sendo armazenado no formato WAV ao longo da realização da prova.

Após o encerramento da prova, o áudio é submetido a um processo de pré-processamento utilizando a ferramenta FFmpeg, que o converte para o formato PCM 16 bits, taxa de amostragem de 16.000 Hz e canal mono, garantindo compatibilidade com o modelo de análise.

A diarização de locutores é realizada pelo modelo *pyannote/speaker-diarization-3.1* [20], disponibilizado pela biblioteca *pyannote.audio*. Esse modelo segmenta o áudio em intervalos de tempo e atribui a cada intervalo o locutor correspondente, respondendo à questão "quem falou e quando?". O resultado é uma linha do tempo de segmentos de fala, com início, fim e rótulo de locutor.

Para demonstrar a eficácia da separação auditiva e a detecção de múltiplos falantes, as Figuras 1 e 2 apresentam o mapeamento de 10 segundos de gravação em dois cenários distintos, com somente um locutor em fala e outro com dois locutores em fala.

A partir dessa segmentação, são calculados: o número total de locutores detectados, o percentual de silêncio e os períodos em que ocorreu alternância rápida entre locutores distintos definida como uma transição em intervalo inferior a 0,5 segundos, indicando possível sobreposição de falas.

Por ser uma tarefa computacionalmente custosa, a análise é executada de forma assíncrona em segundo plano. O professor pode solicitar a análise após o encerramento da prova e consultar o resultado por meio de um *endpoint* REST, que retorna os dados estruturados em JSON contendo o número de locutores, a duração total, o percentual de silêncio, a linha do tempo completa de falas e os períodos suspeitos de múltiplos locutores.

4.5.9 Análise Visual e Estimativa de Pose da Cabeça. A auditoria do foco visual do discente é executada de forma assíncrona após a conclusão do exame. Este processamento em segundo plano garante que a disponibilidade da API principal não seja comprometida por tarefas computacionalmente intensivas. O serviço utiliza o *framework* **MediaPipe** [8], para processar as gravações da *webcam* através do modelo *Face Mesh*. Esta rede neural é capaz de extrair 468 marcos faciais (*landmarks*) tridimensionais por quadro, operando com um limiar mínimo de confiança de 50% para detecção e rastreamento.

Para otimizar a precisão geométrica, o sistema seleciona estrategicamente um subconjunto de seis marcos anatômicos de alta estabilidade: a ponta do nariz (*landmark* 1), os cantos internos dos olhos (*landmarks* 33 e 263), as extremidades da boca (*landmarks* 61 e 291) e o mento (*landmark* 199). Estas coordenadas 2D são mapeadas para um modelo craniano 3D antropométrico (modelo Solvay). A partir desta correspondência, o sistema resolve o problema de geometria projetiva *Perspective-n-Point* (PnP) utilizando a biblioteca **OpenCV**.

O vetor de rotação gerado pelo algoritmo PnP é transformado em matriz de rotação via método de Rodrigues [9] e decomposto em ângulos de Euler por decomposição RQ. O resultado são três medidas de orientação fundamentais: eixo horizontal (*Yaw*), eixo vertical (*Pitch*) e inclinação lateral (*Roll*). A categorização da atenção baseia-se em limiares de 10 graus, classificando o olhar em categorias discretas: frente (*Forward*), esquerda, direita, cima e baixo. Casos em que o rastreamento é perdido são registrados como ausência de face.

A inferência ocorre por amostragem (um a cada cinco quadros) para otimização de recursos. A resiliência é garantida por um mecanismo de ponto de verificação que persiste o estado no banco de dados a cada 500 quadros. O sistema define como "período suspeito" qualquer intervalo contínuo superior a dez segundos em que o olhar esteja desviado da posição frontal, fornecendo um rastro forense para a auditoria docente.

Estrutura de Persistência. Os resultados são armazenados na tabela `video_analyses` do PostgreSQL via *SQLAlchemy ORM*, organizados conforme a Tabela 3.

Tabela 3: Atributos da análise de vídeo.

Grupo	Atributos
Controle	`status`, `total_frames`, `current_frame`, `progress_percentage`
Atenção	`percentage_forward`, `percentage_left`, `percentage_right`, `percentage_up`, `percentage_down`, `percentage_no_face`
Distração	`total_position_changes`, `position_changes_per_minute`
Suspeitos	`suspicious_periods`

Fonte: Elaborado pelo autor (2026).

Estrutura de Resposta da API. A rota da API de resultados retorna um objeto JSON contendo as métricas de distribuição e os períodos suspeitos, conforme ilustrado no Código 1.

Código 1: Estrutura JSON de resposta da análise visual.

```
{
  "status": "completed",
  "prova_id": int,
  "aluno_id": int,
  "metrics": {
    "percentage_forward": float,
    "percentage_left": float,
    "percentage_right": float,
    "percentage_up": float,
    "percentage_down": float,
    "percentage_no_face": float,
    "position_changes_per_minute": float,
    "total_position_changes": int
  },
  "suspicious_periods": [
    {
      "inicio": float,
      "fim": float,
      "duracao": float,
      "direcao": "Esquerda" | "Direita" | "Cima" | "Baixo
          "
    }
  ],
  "completed_at": "ISO 8601"
}
```

4.5.10 Detecção de Similaridade e Plágio de Código. A verificação de autoria das soluções submetidas é realizada por um serviço automatizado de detecção de similaridade, que executa comparações par a par entre os códigos enviados para uma mesma questão. A análise é processada por avaliação, sendo possível a reexecução sob demanda pelo docente, o que atualiza os resultados previamente armazenados. Para garantir a consistência estatística, as submissões são agrupadas pelo identificador `questao_id`, assegurando que a comparação ocorra apenas entre discentes que resolveram o mesmo problema.

O cálculo de similaridade fundamenta-se no algoritmo de *Ratcliff/Obershelp*, utilizando a implementação da classe `difflib.SequenceMatcher` da linguagem Python. Para cada par de submissões (s_1, s_2), o sistema gera duas métricas percentuais distintas:

(1) **Similaridade Textual:** calculada sobre o código-fonte original, capturando repetições literais e de formatação;
(2) **Similaridade Normalizada:** aplicada sobre uma representação tratada do código, visando identificar plágios que utilizam variações superficiais.

A etapa de normalização utiliza um analisador sintático baseado em expressões regulares, adaptado para a linguagem Python. O processo consiste nas seguintes transformações:

- Remoção de comentários de linha (`#`, `//`) e de bloco (`/* */`);
- Exclusão de *docstrings* (`"` ou `"'`) na linguagem Python;
- Colapso de múltiplos espaços em branco e remoção de linhas vazias;
- Substituição de literais de texto (*strings*) pelo marcador de posição `"STR"`;
- Substituição de literais numéricos (inteiros e pontos flutuantes) pelo marcador de posição `NUM`.

Ressalta-se que a normalização proposta não realiza a renomeação de identificadores ou variáveis. Consequentemente, a métrica é eficaz na detecção de cópias com alterações de formatação e comentários, mas possui limitações contra técnicas de refatoração sistemática de nomes.

$$S_{\max} = \max(s_{\text{textual}}, s_{\text{normalizada}})$$

Pares que atingem um limiar configurável (tipicamente $\geq 70\%$) são sinalizados com o atributo `flagged: true`. Os resultados são apresentados ao docente em ordem decrescente de similaridade, priorizando os casos críticos. A estrutura de dados retornada ao *frontend*, contendo os metadados das questões e os detalhes dos pares suspeitos, é detalhada no Código 2.

Código 2: Estrutura JSON da resposta do serviço de similaridade.

```
{
  "questions": [
    {
      "questao_id": int,Painel
      "questao_titulo": string,
      "total_submissions": int,
      "total_pairs": int,
      "flagged_pairs": int,
      "pairs": [
        {
          "aluno_a": { "id": int, "nome": string, "email
              ": string },
          "aluno_b": { "id": int, "nome": string, "email
              ": string },
          "text_similarity": float,
          "normalized_similarity": float,
          "max_similarity": float,
          "flagged": boolean
        }
      ]
    }
  ],
  "total_students": int,
  "total_flagged_pairs": int,
  "threshold_used": float
}Painel
```

Cabe ressaltar que o sistema atua como uma ferramenta de auxílio à decisão; a deliberação final sobre a ocorrência de plágio é de responsabilidade exclusiva do docente.

4.5.11 Biometria Comportamental por Dinâmica de Digitação. O serviço de dinâmica de digitação realiza a extração de características comportamentais a partir dos eventos de teclado capturados pelo editor de código na interface do usuário. A fundamentação teórica deste módulo baseia-se na pesquisa de Santos Filho [11], que propôs a identificação de comportamentos ilegítimos utilizando a técnica de *Keystroke Dynamics*. A arquitetura desenvolvida neste projeto adapta essa metodologia para um ecossistema cliente-servidor, onde os eventos são capturados de forma contínua no navegador e processados centralizadamente no servidor.

A coleta dos dados rítmicos ocorre na camada de apresentação, utilizando as APIs nativas do editor Monaco (`onKeyDown` e `onKeyUp`). Cada interação é modelada como um objeto contendo a identificação da tecla (`key`), o carimbo de tempo (`timestamp`) e o tipo de ação (`command_type`). A partir desses dados, o sistema deriva duas métricas fundamentais: o tempo de retenção (*hold time*), definido pela diferença entre o pressionamento e a soltura de uma

tecla, e o tempo de voo (*flight time*), que mede o intervalo entre teclas consecutivas.

Devido às restrições de *sandbox* do navegador, os eventos são categorizados entre digitação regular e comandos suspeitos (como colagens, retrocessos massivos ou movimentações bruscas do cursor). Para garantir a resiliência, os pacotes de eventos são transmitidos ao servidor em lotes a cada dois minutos e persistidos em formato JSON no banco de dados PostgreSQL.

O processamento analítico no servidor ocorre de forma assíncrona em duas etapas operacionais. Inicialmente, estabelece-se a linha de base biométrica do estudante por meio do cálculo de médias e desvios padrão dos tempos de retenção e voo. Em seguida, o algoritmo quantifica a disparidade rítmica durante o uso da área de transferência aplicando a Distância de Manhattan. O cálculo é realizado entre pares de eventos adjacentes, somando as diferenças absolutas da variação temporal (Δt), do tempo de retenção (Δh) e do tempo de voo (Δf):

$$d_{seq} = \Delta t + |\Delta h| + |\Delta f| \tag{1}$$

A média dessas distâncias ao longo da avaliação compõe uma métrica de irregularidade rítmica agregada. O dado analítico final disponibilizado ao docente consolida o perfil temporal da sessão e a frequência de comandos evasivos, oferecendo um rastro legível e embasado para a auditoria pedagógica.

4.6 Privacidade e Tratamento de Dados (LGPD)

O desenvolvimento de uma plataforma de *proctoring* que lida com dados biométricos e comportamentais como vídeo, áudio, e dinâmica de digitação impõe uma responsabilidade ética e legal sobre o tratamento dessas informações. Embora a implementação atual se concentre na validação técnica dos módulos de análise, o projeto prevê, para uma futura implantação em ambiente de produção, a adoção de uma política de privacidade estrita e alinhada com os princípios da Lei Geral de Proteção de Dados (LGPD, Lei nº 13.709/2018).

A política de tratamento de dados será fundamentada em três pilares:

- **Criptografia de Dados em Repouso:** Todos os arquivos que contenham informações de identificação pessoal, especialmente os artefatos de mídia (gravações de tela, áudio e webcam), serão armazenados em formato criptografado. Isso garante que o acesso não autorizado ao sistema de arquivos do servidor não comprometa a privacidade dos discentes.
- **Política de Retenção e Expurgo:** Será definida uma política de retenção de dados com prazo máximo, alinhado às necessidades acadêmicas (e.g., o final do período letivo somado ao prazo para recursos de avaliação). Após esse período, todos os dados sensíveis vinculados à identidade do aluno serão permanentemente removidos ou anonimizados, em conformidade com o princípio da necessidade.
- **Acesso Restrito e Auditável:** O acesso aos dados brutos de monitoramento será restrito exclusivamente ao docente responsável pela disciplina e para a finalidade específica de auditoria da avaliação, em concordância com as bases legais de legítimo interesse e execução de contrato (educacional).

Dessa forma, busca-se equilibrar a necessidade de garantir a integridade acadêmica com o direito fundamental à privacidade dos estudantes, posicionando a plataforma não como uma ferramenta de vigilância contínua, mas como um mecanismo de suporte à decisão transparente e responsável.

5 Resultados

A validação do sistema baseou-se em cenários de uso controlado, simulando comportamentos típicos de exames remotos para avaliar a sensibilidade dos algoritmos de monitoramento.

5.1 Fluxo de Execução e Interface do Usuário

A validação das interfaces e do fluxo de trabalho da plataforma foi realizada através da simulação de um ciclo completo de avaliação, abrangendo desde a configuração docente até a entrega de resultados ao discente. O cenário de teste consistiu na criação de uma avaliação técnica contendo uma questão de lógica básica (soma de dois números).

5.1.1 Configuração do Exame e Questões. O ciclo inicia-se na área administrativa do professor, onde foi criado o exame intitulado "Prova Exemplo". Nesta etapa, definiu-se a descrição da atividade e o vínculo do discente através do e-mail institucional (Figura 21). Na sequência, configurou-se a questão "Faça a soma de dois números", com peso de 100% da nota. O sistema permite a inserção de casos de exemplo (entrada e saída esperada), (Figura 22).

5.1.2 Gerenciamento e Acesso ao Exame. Após a publicação, o sistema disponibiliza um *card* de controle indicando que a prova está em andamento (Figura 23). Simultaneamente, no painel do aluno, a avaliação torna-se visível, permitindo o acesso ao ambiente monitorado de execução (Figura 24).

5.1.3 Ambiente de Execução e Submissão. Ao iniciar o exame, o discente é conduzido ao editor de código integrado, onde as ferramentas de *proctoring* (câmera, áudio e tela) operam de forma transparente (Figura 25). Após a conclusão e submissão do código, o docente pode acessar a página de detalhes do exame para listar todas as entregas realizadas (Figura 26).

5.1.4 Avaliação e Feedback. O sistema permite a revisão detalhada de cada submissão, exibindo o código-fonte enviado e a pontuação final calculada a partir da avaliação individual de cada questão (Figura 27). Por fim, o ciclo se encerra com a disponibilização do resultado ao aluno, que pode visualizar seu desempenho e o status da sua entrega diretamente em seu perfil (Figura 28).

5.2 Monitoramento Visual e Gaze Tracking

A validação da estimativa de pose da cabeça foi realizada através de três cenários experimentais projetados para testar a precisão, a robustez e a detecção de ausência do discente.

5.2.1 Teste de Direcionamento em Cinco Posições. No primeiro cenário, foi realizado um teste de posicionamento fixo onde o usuário permaneceu por cinco segundos em cada uma das cinco direções fundamentais: frente, esquerda, direita, cima e baixo. Conforme apresentado na Figura 30, o sistema classificou as transições de forma parcialmente correta.

Observou-se um viés sistemático para a zona de visão à direita, mesmo durante intervalos de foco em outras direções. Além disso, a métrica de frequência de troca de posição apresentou valores anomalamente elevados. Esse comportamento evidencia uma instabilidade no vetor de decisão do algoritmo, que interpreta pequenas variações rítmicas ou ruídos na captura dos marcos faciais como mudanças de estado, reduzindo a confiabilidade da métrica de alternância de olhar.

5.2.2 Robustez à Rotação do Dispositivo. O segundo cenário testou a robustez do modelo frente a distorções no ângulo da câmera. O experimento consistiu em rotacionar o dispositivo (webcam) em aproximadamente 30 graus nos sentidos horário e anti-horário, mantendo o olhar fixo para o lado esquerdo.

Como demonstrado na Figura 31, o algoritmo apresentou resiliência satisfatória. Apesar da inclinação acentuada do plano da imagem, que poderia deslocar o ponto de referência central, a detecção dos desvios laterais permaneceu estável, confirmando a eficácia.

5.2.3 Detecção de Ausência de Face. Por fim, testou-se a capacidade do serviço em identificar a ausência do candidato na cena. O teste simulou o início de uma prova sem a presença de um rosto detectável pelo *MediaPipe*. O sistema registrou corretamente o estado de "Ausência de Face"(Figura 32), o que é crucial para identificar situações em que o aluno se retira do ambiente de prova.

5.3 Diarização de Áudio

A eficácia do serviço de análise acústica foi avaliada por meio de dois cenários experimentais distintos, visando validar a capacidade do modelo *pyannote.audio* em segmentar e identificar locutores em ambiente de avaliação.

5.3.1 Cenário Monolocutor. No primeiro experimento, o discente realizou a leitura de uma questão do exame em voz alta, sem interferências externas. O objetivo era validar se o sistema manteria a consistência de um único identificador de locutor durante toda a sessão. Conforme ilustrado na Figura 33, os resultados confirmaram a precisão do algoritmo, que classificou corretamente a totalidade da atividade vocal como pertencente a um único usuário, sem a geração de falsos positivos de colagem sonora.

5.3.2 Cenário de Múltiplos Locutores. O segundo cenário simulou um ambiente de colaboração não autorizada. Para este teste, o usuário realizou a leitura da questão simultaneamente à reprodução de áudio externo proveniente de vídeos da plataforma *YouTube*, contendo o diálogo de diversas pessoas. O objetivo era testar a sensibilidade do modelo na detecção de pluralidade vocal.

Os resultados, apresentados na Figura 34, demonstram que o sistema identificou com sucesso a existência de mais de um falante, disparando corretamente o alerta de irregularidade. Entretanto, observou-se uma limitação na granularidade da classificação: embora o áudio de fundo contivesse múltiplas vozes, o modelo convergiu para a identificação de apenas dois locutores distintos.

Essa convergência pode ser atribuída à sobreposição de frequências (*overlap*) e à baixa qualidade da captura do áudio secundário pela *webcam*, o que levou o modelo a agrupar diferentes vozes externas em um único perfil biométrico genérico. Apesar dessa limitação na contagem exata, a funcionalidade principal de alertar o docente sobre a presença de vozes alheias ao discente foi plenamente atendida.

5.4 Dinâmica de Digitação (Keystroke)

A avaliação da biometria comportamental baseou-se na análise do ritmo de digitação e na detecção de eventos de área de transferência. Para validar o modelo, foram estabelecidos três cenários experimentais, utilizando como base a implementação do algoritmo *Bubble Sort*.

5.4.1 Cenário de Linha de Base (Baseline). O primeiro cenário consistiu na resolução autoral da questão por um discente, simulando um fluxo de trabalho real sem consulta externa. Durante a implementação, o sistema registrou um volume de 301 eventos de teclado, resultando em um índice de suspeita de 35,0% (Figura 35).

Esta pontuação, embora relevante, reflete as hesitações naturais, pausas para raciocínio e correções de sintaxe típicas de um processo de desenvolvimento genuíno. O elevado número de eventos indica uma interação intensa com o editor, onde o código é construído de forma incremental.

5.4.2 Cenário de Inserção por Área de Transferência. O segundo teste simulou o plágio via transposição de blocos prontos. O algoritmo atribuiu um índice de suspeita de 46,3%, detectando a ocorrência de comandos *Ctrl+C* e *Ctrl+V* (Figura 36).

Embora a pontuação seja alta, a análise pedagógica deve considerar que o uso da área de transferência é comum para reorganização de código próprio. Contudo, o baixo volume de eventos de digitação em comparação ao bloco final de código inserido é um forte indicativo de que a solução não foi construída dentro do ambiente monitorado.

5.4.3 Cenário de Digitação com Cópia Manual. No terceiro experimento, simulou-se a cópia caractere a caractere de uma fonte externa. Este cenário resultou em um índice de 29,2% de suspeita (Figura 37). O sistema detectou uma regularidade rítmica incomum, mas a pontuação reduzida demonstra a dificuldade em distinguir uma cópia manual de uma digitação pausada apenas por métricas de tempo de voo (*flight time*).

Com base nos resultados obtidos, observa-se que o módulo de *keystroke* é eficiente para identificar o plágio imediato, mas a comparação entre os cenários revela um padrão crucial relacionado ao volume total de eventos. No cenário de linha de base, a resolução real do problema gerou 301 eventos, um número significativamente superior aos cenários de plágio, onde o código é inserido de forma integral ou transcrito com poucas correções.

Essa observação sugere uma nova heurística para o cálculo de suspeita: a correlação entre o tamanho final do código e o número de eventos gerados. Um código extenso com baixo número de interações sugere que a solução foi concebida fora do editor monitorado. Assim, a dinâmica de digitação não deve apenas avaliar o ritmo temporal, mas também a densidade de construção da solução. Essa percepção reforça a necessidade do rastro multimodal, no qual o baixo volume de eventos no teclado, somado a desvios detectados pelo *Gaze Tracking*, fornece ao docente uma evidência robusta de que a autoria não ocorreu de forma legítima durante o exame.

5.5 Detecção de Plágio e Similaridade

Para validar o serviço de similaridade, foram realizados experimentos comparativos utilizando quatro implementações distintas do algoritmo *Bubble Sort*. O objetivo foi avaliar o comportamento do sistema frente a códigos genuinamente independentes e códigos resultantes de plágio com técnicas de ofuscação superficial.

5.5.1 Cenário 1: Implementações Independentes. Neste cenário, comparou-se a solução do Aluno 1 (Código 3), que utiliza uma abordagem clássica com `range`, e a do Aluno 2 (Código 4), que emprega uma *flag* de otimização e a estrutura `while`.

Código 3: Aluno 1: Implementação Independente A.

```
def bubble_sort(lista):
    n = len(lista)
    for i in range(n):
        for j in range(0, n - i - 1):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1],
                    lista[j]
    return lista
```

Código 4: Aluno 2: Implementação Independente B.

```
def ordenar_bolha(dados):
    tamanho = len(dados)
    houve_troca = True
    passagem = 0

    while houve_troca:
        houve_troca = False
        for k in range(tamanho - passagem - 1):
            if dados[k] > dados[k+1]:
                dados[k], dados[k+1] = dados[k+1], dados[
                    k]
                houve_troca = True
        passagem += 1
    return dados
```

Como esperado para códigos desenvolvidos de forma independente, o sistema apresentou baixos índices de similaridade, com 25,0% na métrica textual e 15,3% na normalizada (Figura 38). O *status* foi classificado corretamente como "OK", indicando a ausência de plágio devido às diferenças estruturais na lógica de controle.

5.5.2 Cenário 2: Plágio com Ofuscação Superficial. O segundo teste utilizou o Código 5 como fonte original e o Código 6 como uma versão modificada superficialmente, na qual houve a renomeação de variáveis, alteração de comentários e modificação da técnica de troca de valores.

Código 5: Aluno 3: Fonte do Plágio.

```
def executar_bubble(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                temp = arr[j]
                arr[j] = arr[j + 1]
                arr[j + 1] = temp
    return arr
```

Código 6: Aluno 4: Plágio Maquiado.

```
def executar_bubble(vetor):
    limite = len(vetor)
    for x in range(limite):
        for y in range(limite - x - 1):
            if vetor[y] > vetor[y + 1]:
                # Inverte os valores sem usar variavel
                    temporaria
                vetor[y], vetor[y + 1] = vetor[y + 1],
                    vetor[y]
    return vetor
```

Neste caso, os resultados apontaram 42,0% de similaridade textual e 20,0% de similaridade normalizada (Figura 39). Embora o sistema tenha detectado uma correlação parcial, o *status* permaneceu como "OK", uma vez que os valores ficaram abaixo do limiar configurado de 70%.

Com base nos resultados achados, observa-se que o sistema foi eficaz em validar a legitimidade de códigos independentes, mas apresentou limitações no cenário de plágio maquiado. A similaridade normalizada de apenas 20% no segundo teste revela que a técnica de normalização atual, baseada estritamente em expressões regulares sem a renomeação de identificadores, é sensível a variações de nomenclatura de variáveis e pequenas mudanças estruturais (como a substituição de uma variável temporária por atribuição múltipla).

Embora os códigos possuam lógica idêntica, a baixa pontuação normalizada indica que o algoritmo de *SequenceMatcher* prioriza a ordem exata dos caracteres remanescentes. Isso demonstra que, para o sistema atingir a confiabilidade esperada, seria necessária uma heurística de normalização mais agressiva, como a tokenização de identificadores, para garantir que mudanças superficiais de nomes não mascarem a identidade lógica das submissões.

6 Discussão

A análise dos resultados experimentais demonstra que, embora cada módulo de monitoramento possua valor individual, abordagens baseadas em um único vetor de análise são insuficientes para uma auditoria de integridade robusta. A verdadeira força da plataforma reside na correlação de dados multimodais, onde a combinação de diferentes sinais permite a construção de uma evidência coesa para o suporte à decisão docente.

Individualmente, cada módulo apresentou limitações que reforçam essa conclusão. O algoritmo de estimativa de pose facial (conhecido como *gaze tracking*), por exemplo, mostrou-se instável para determinar a direção exata do olhar, mas provou ser um excelente sensor de presença, identificando com precisão a ausência do discente. Da mesma forma, a diarização de áudio cumpriu seu objetivo principal ao detectar a presença de vozes múltiplas, um forte indício de colaboração, ainda que sua capacidade de contar o número exato de locutores em ambientes com sobreposição sonora seja limitada.

No monitoramento da codificação, a análise da dinâmica de digitação revelou que a métrica mais relevante não é apenas o ritmo, mas a densidade de eventos. Uma resolução autoral legítima gerou um volume de interações (301 eventos) muito superior aos cenários de cópia, onde o código surge no editor com pouca atividade. Este baixo volume de digitação, quando correlacionado com outros indícios como o desvio de olhar detectado pelo módulo visual ou a similaridade de código com outro colega , transforma uma suspeita fraca em uma evidência forte de que a solução não foi concebida no ambiente da prova.

A detecção de similaridade, por sua vez, foi eficaz para identificar cópias literais, mas demonstrou fragilidade diante de ofuscações simples, como a renomeação de variáveis. Uma similaridade normalizada de apenas 20% em códigos com lógica idêntica evidencia a necessidade de evoluir para análises sintáticas mais profundas, como a comparação de Árvores de Sintaxe Abstrata (AST), para tornar o sistema imune a alterações cosméticas.

Portanto, a principal contribuição deste trabalho não é a criação de um juiz algorítmico infalível, mas sim a orquestração de um painel de auditoria. O sistema atinge seu objetivo ao fornecer um conjunto de sinais que, juntos, pintam um quadro claro do comportamento do aluno. A suspeita de ausência de um aluno, por si só, é apenas um dado; contudo, a ausência detectada pelo módulo de vídeo, somada à inserção de um bloco de código via área de transferência (detectado pelo módulo de digitação) e à alta similaridade deste código com o de outro aluno, constitui uma base de evidências concreta para a ação do professor.

Finalmente, a capacidade de reproduzir integralmente a sessão de prova, sincronizando as gravações de áudio, vídeo da *webcam* e da área de trabalho, permanece como o pilar fundamental da auditoria. Este rastro completo garante que a decisão final seja humana, informada e justa, permitindo ao docente correlacionar os alertas automáticos com o comportamento real e consolidando a plataforma como uma ferramenta de suporte à decisão transparente e responsável.

7 Trabalhos Futuros

Para a evolução da plataforma e a superação das limitações técnicas identificadas nesta pesquisa, propõem-se as seguintes frentes de desenvolvimento, organizadas por área de atuação:

Visão Computacional e Ambiente de Prova.

- **Aprimoramento do Rastreamento Ocular:** Substituir a atual estimativa por pose da cabeça por algoritmos de rastreamento ocular baseados na detecção da pupila. Essa abordagem tende a oferecer maior precisão na identificação do foco visual, reduzindo os ruídos gerados por variações de iluminação e pequenos movimentos da cabeça.
- **Mitigação de Restrições do Navegador:** Para superar as limitações do ambiente restrito (*sandbox*) do navegador, duas estratégias podem ser exploradas:
  (1) O desenvolvimento de um navegador customizado (*fork*) que conceda permissões granulares de acesso, como a visualização de múltiplas abas e o monitoramento de eventos do sistema.
  (2) A criação de um agente de software local (binário) que, uma vez instalado, teria acesso direto aos dispositivos, eventos de teclado em nível de sistema operacional e informações sobre o uso de múltiplos monitores, fornecendo um monitoramento mais completo.

Análise de Código-Fonte.

- **Suporte a Múltiplas Linguagens:** Expandir a arquitetura para suportar múltiplos executores de código (*runners*), permitindo que a plataforma avalie uma gama maior de linguagens de programação, além de Python.
- **Detecção de Plágio Robusta:** Evoluir o serviço de similaridade para uma abordagem baseada em Árvores de Sintaxe Abstrata (AST). Esta técnica analisa a estrutura fundamental do código, tornando a detecção de plágio imune a ofuscações superficiais, como renomeação de variáveis, alteração de comentários ou troca de estruturas de laço (e.g., `for` por `while`).

Biometria Comportamental.

- **Análise Preditiva da Dinâmica de Digitação:** Aprimorar o módulo de dinâmica de digitação com modelos de aprendizado de máquina (*Machine Learning*). Tais modelos poderiam ser treinados para correlacionar o volume de eventos de teclado com a complexidade ciclomática do código produzido. Essa heurística permitiria diferenciar com maior acurácia a digitação fluida de um desenvolvedor experiente da transcrição manual lenta, que caracteriza a cópia de uma solução preexistente.

Validação e Desempenho.

- **Avaliação Experimental em Ambiente Real:** Conduzir avaliações experimentais controladas, seguidas de validação empírica em um ambiente de prova real com usuários. Esta etapa é fundamental para testar o desempenho arquitetural e a escalabilidade da plataforma sob carga, além de aferir rigorosamente a eficácia dos módulos de detecção propostos e a estabilidade do sistema durante a aplicação prática de exames.

8 Considerações Finais

Este trabalho apresentou o desenvolvimento e a validação de uma plataforma de *proctoring* multimodal que prioriza a transparência e o suporte ao docente no processo de avaliação remota. Ao integrar análises de vídeo, áudio, dinâmica de digitação e similaridade de código em uma arquitetura de software coesa, o sistema propõe uma solução de código aberto para uma demanda crescente: a necessidade de ferramentas que garantam a integridade acadêmica sem abdicar da indispensável auditoria humana.

A pesquisa demonstrou a tese central de que, embora algoritmos de análise isolados apresentem limitações, a convergência de múltiplas fontes de dados gera um rastro forense robusto e confiável. O projeto cumpriu seus objetivos ao entregar não apenas um sinalizador automático de irregularidades, mas uma interface de revisão que capacita o professor com dados concretos. Dessa forma, consolida-se uma ferramenta que permite ao ambiente de ensino a distância da UFCG, e outras instituições, realizar avaliações de forma mais justa, transparente e tecnologicamente embasada.

Referências

[1] CHIAVENATO, I. **Recursos Humanos**: o capital humano das organizações. 9. ed. rev. e atual. Rio de Janeiro: Elsevier, 2009. Disponível em: https://www.academia.edu/36837455/Recursos_Humanos. Acesso em: 20 fev. 2026.

[2] KOCDAR, S.; KARADENIZ, A.; BOZKURT, A. et al. **Cheating and plagiarism in e-assessment**. Open Praxis, v. 10, n. 3, 2018. Disponível em: https://files.eric.ed.gov/fulltext/EJ1197094.pdf. Acesso em: 10 fev. 2026.

[3] DAWSON, Phillip. **Defending assessment security in a digital world**. Assessment & Evaluation in Higher Education, 2019. Disponível em: https://api.pageplace.de/preview/DT0400.9781000201000_A39737518/preview-9781000201000_A39737518.pdf. Acesso em: 23 fev. 2026.

[4] NIGAM, Anurag et al. **A systematic review on AI-based proctoring systems**. Education and Information Technologies, 2021. Disponível em: https://pubmed.ncbi.nlm.nih.gov/34177348/. Acesso em: 23 jan. 2026.

[5] DENDIR, Serkan; MAXWELL, Randy. **Cheating in online courses: Evidence from online proctoring**. Computers in Human Behavior Reports, v. 2, 2020. Disponível em: https://www.sciencedirect.com/science/article/pii/S2451958820300336. Acesso em: 23 jan. 2026.

[6] ALESSIO, H. M. et al. **Examining the effect of proctoring on online test scores**. Online Learning Journal, v. 21, n. 1, 2017. Disponível em: https://courses.up.eku.edu/IDCWS/articles/Examining%20the%20Effect%20of%20Proctoring%20on%20Online%20Test%20Scores.pdf. Acesso em: 2 fev. 2026.

[7] SLUSKY, L. **Cybersecurity of Online Proctoring Systems**. Journal of International Technology and Information Management, Vol. 29, Iss. 1, Article 3, 2020. Disponível em: https://scholarworks.lib.csusb.edu/jitim/vol29/iss1/3/. Acesso em: 15 fev. 2026.

[8] LUGARESI, Camillo et al. **MediaPipe: a framework for building perception pipelines**. arXiv preprint arXiv:1906.08172, 2019. Disponível em: https://arxiv.org/abs/1906.08172. Acesso em: 2 fev. 2026.

[9] OPENCV. **Camera Calibration and 3D Reconstruction – calib3d Module Documentation**. OpenCV.org, [s.d.]. Disponível em: https://docs.opencv.org/3.4/d9/d0c/group__calib3d.html#ga61585db663d9da06b68e70cfbf6a1eac. Acesso em: 15 fev. 2026.

[10] RATCLIFF, John W.; OBERSHELP, John A. **Pattern matching: the Gestalt approach**. Dr. Dobb's Journal, 1988. Disponível em: https://web.archive.org/web/20140617171654/https://www.drdobbs.com/database/pattern-matching-the-gestalt-approach/184407970?pgno=5. Acesso em: 3 fev. 2026.

[11] SANTOS FILHO, R. P. dos. **Identificação de comportamento ilegítimo em atividades educacionais de programação**. 2025. 43 f. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) – Centro de Engenharia Elétrica e Informática, Universidade Federal de Campina Grande, Campina Grande, 2025.

[12] LEWIS, J.; FOWLER, M. **Microservices: a definition of this new architectural term**. 2014. Disponível em: https://martinfowler.com/articles/microservices.html. Acesso em: 2 fev. 2026.

[13] HACKERRANK. **HackerRank – Technical Hiring Platform**. HackerRank.com, [s.d.]. Disponível em: https://www.hackerrank.com. Acesso em: 2 fev. 2026.

[14] WECP. **Sherlock AI Agent – Proctoring Feature**. WeCreateProblems.com, [s.d.]. Disponível em: https://www.wecreateproblems.com/sherlock. Acesso em: 2 fev. 2026.

[15] UTKRUSHT. **Utkrusht – Technical Assessments to Hire Developers**. Utkrusht.ai, [s.d.]. Disponível em: https://utkrusht.ai/. Acesso em: 12 dez. 2025.

[16] SAFEEXAMBROWSER. **Safe Exam Browser – News**. SafeExamBrowser.org, [s.d.]. Disponível em: https://safeexambrowser.org/news_en.html. Acesso em: 17 fev. 2026.

[17] JUDGE0. **Code execution made simple for every business**. Judge0 Official Documentation, 2025. Disponível em: https://judge0.com/. Acesso em: 17 fev. 2026.

[18] NAPOLITANO, K. **Self-hosting Judge0: a step-by-step guide using Amazon EC2, Lambda, and S3**. Tutorials Dojo, 2026. Disponível em: https://tutorialsdojo.com/self-hosting-judge0-a-step-by-step-guide-using-aws-ec2-lambda-and-s3/. Acesso em: 17 fev. 2026.

[19] SAFE EXAM BROWSER. **About: overview**. Safe Exam Browser Project, 2025. Disponível em: https://safeexambrowser.org/about_overview_en.html. Acesso em: 12 dez. 2025.

[20] PLAQUET, Alexis; BREDIN, Hervé. **Powerset multi-class cross entropy loss for neural speaker diarization**. Interspeech, 2023. Disponível em: https://arxiv.org/abs/2310.13025. Acesso em: 5 fev. 2026.

[21] HACKERRANK. **Can Proctor Mode Really Detect ChatGPT? Inside HackerRank's 2025 AI Plagiarism Engine**. HackerRank Blog, 2025. Disponível em: https://www.hackerrank.com/writing/can-proctor-mode-detect-chatgpt-hackerrank-2025-ai-plagiarism-engine. Acesso em: 5 fev. 2026.

[22] KAUSHIK, A. **How proctoring platforms detect and prevent cheating?**. WeCP Blog, 16 fev. 2026. Disponível em: https://www.wecreateproblems.com/blog/how-proctoring-platforms-detect-and-prevent-cheating. Acesso em: 5 fev. 2026.

[23] NOMAN, T. **Top 10 Codility alternatives for technical assessments in 2026**. TestTrick Blog, 9 jan. 2026. Disponível em: https://www.testtrick.com/blogs/codility-alternatives. Acesso em: 25 jan. 2026.

[24] YANG, S. **Codility pricing (2025): plans, hidden limits, and better prep options**. ShadeCoder, 5 dez. 2025. Disponível em: https://www.shadecoder.com/blogs/codility-pricing-2025. Acesso em: 23 jan. 2026.

[25] QUILGO TEAM. **Quilgo® proctoring for Moodle**. Moodle Plugins Directory, [s.d.]. Disponível em: https://moodle.org/plugins/quizaccess_quilgo. Acesso em: 10 fev. 2026.

A Catálogo de Figuras da Plataforma

A.1 Interfaces do Sistema

A.2 Análises e Relatórios

A.3 Modelos C4

A.4 Resultados Experimentais – Fluxo de Execução

A.5 Resultados Experimentais – Monitoramento Visual

A.6 Resultados Experimentais – Diarização de Áudio

A.7 Resultados Experimentais – Dinâmica de Digitação

A.8 Resultados Experimentais – Detecção de Similaridade

A.9 Figuras Complementares

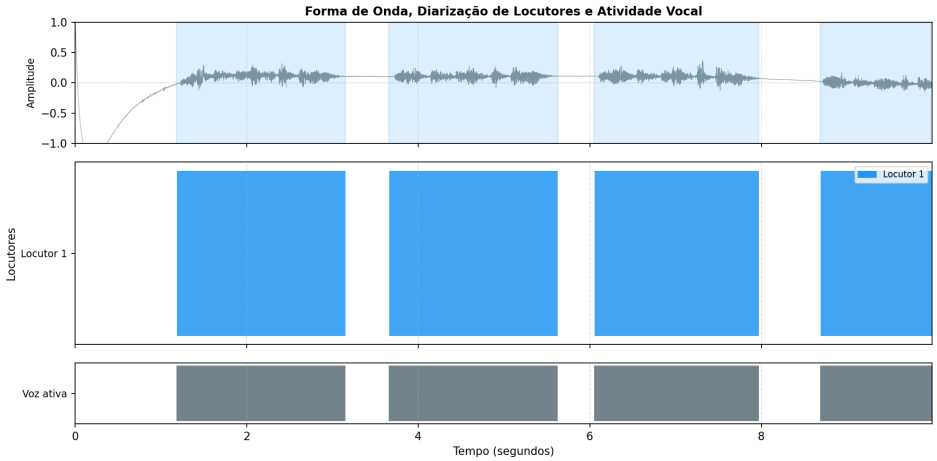


Figura 1: Exemplo de análise de áudio contendo apenas um locutor durante o intervalo de 10 segundos.

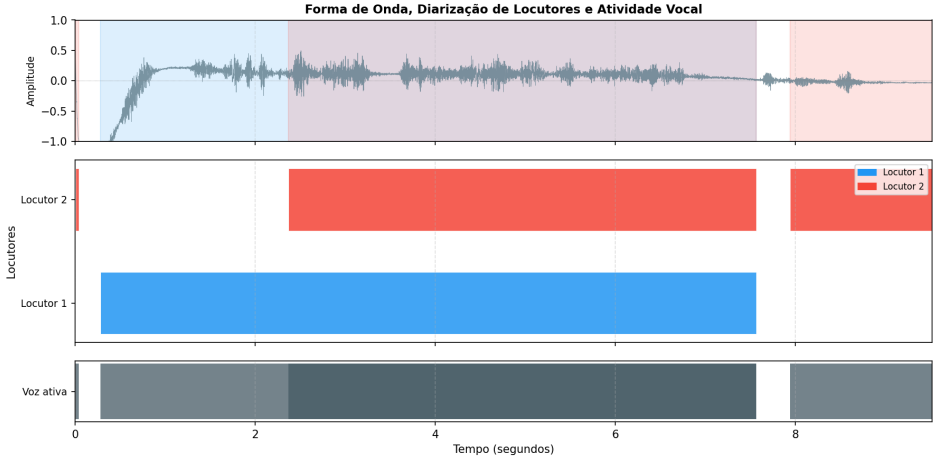


Figura 2: Exemplo de análise de áudio com a detecção de dois locutores distintos no mesmo intervalo, indicando interação.

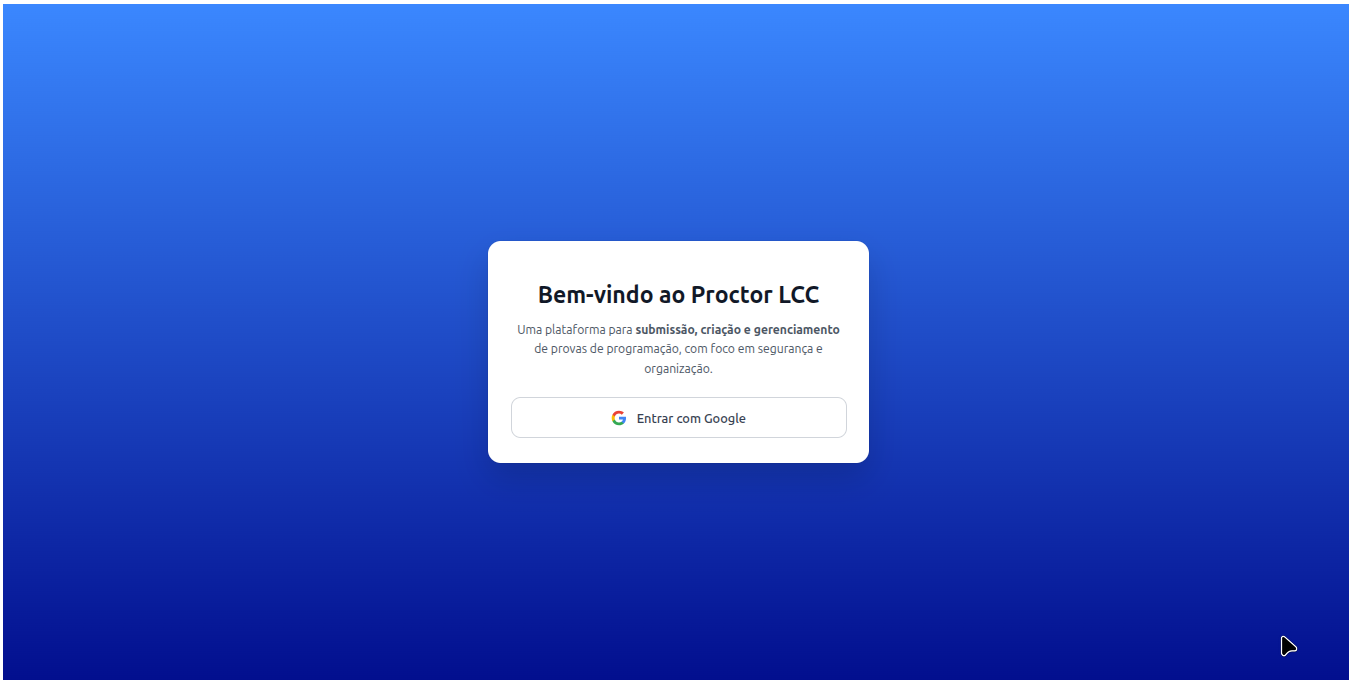


Figura 3: Interface de Login com integração Google OAuth.

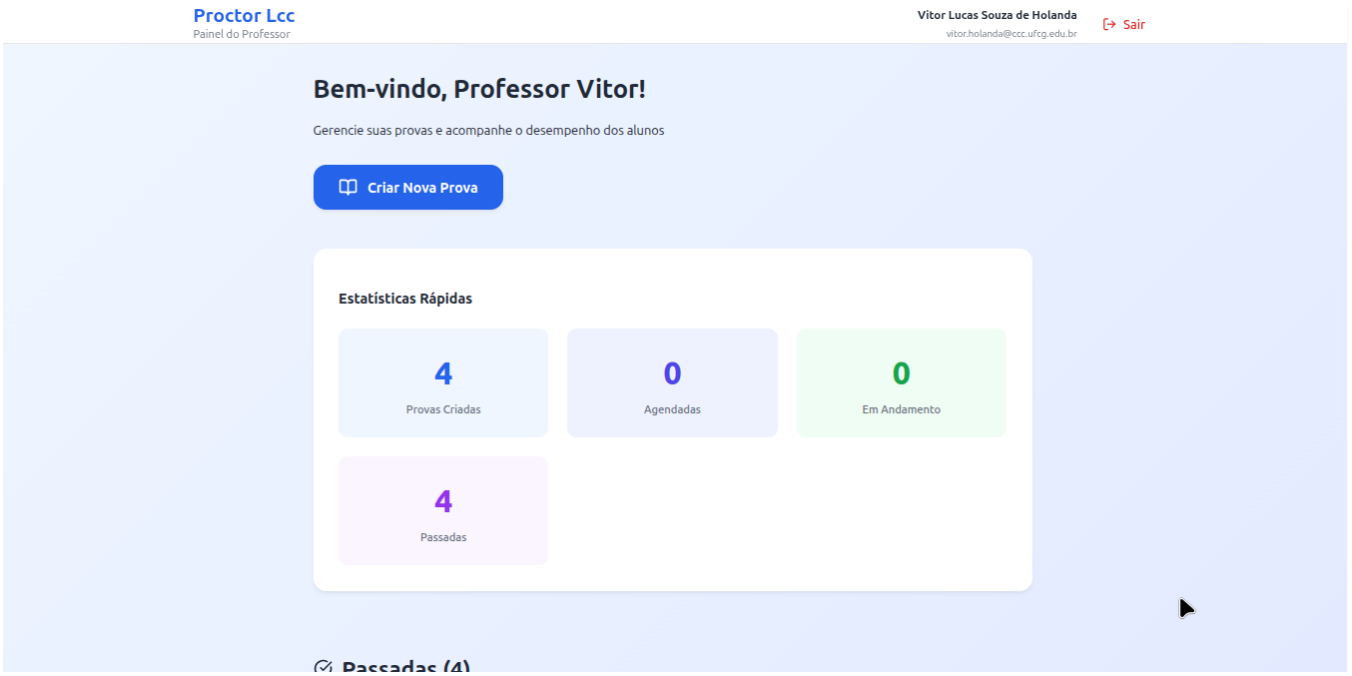


Figura 4: Painel do Professor: Gestão de exames e status de monitoramento.

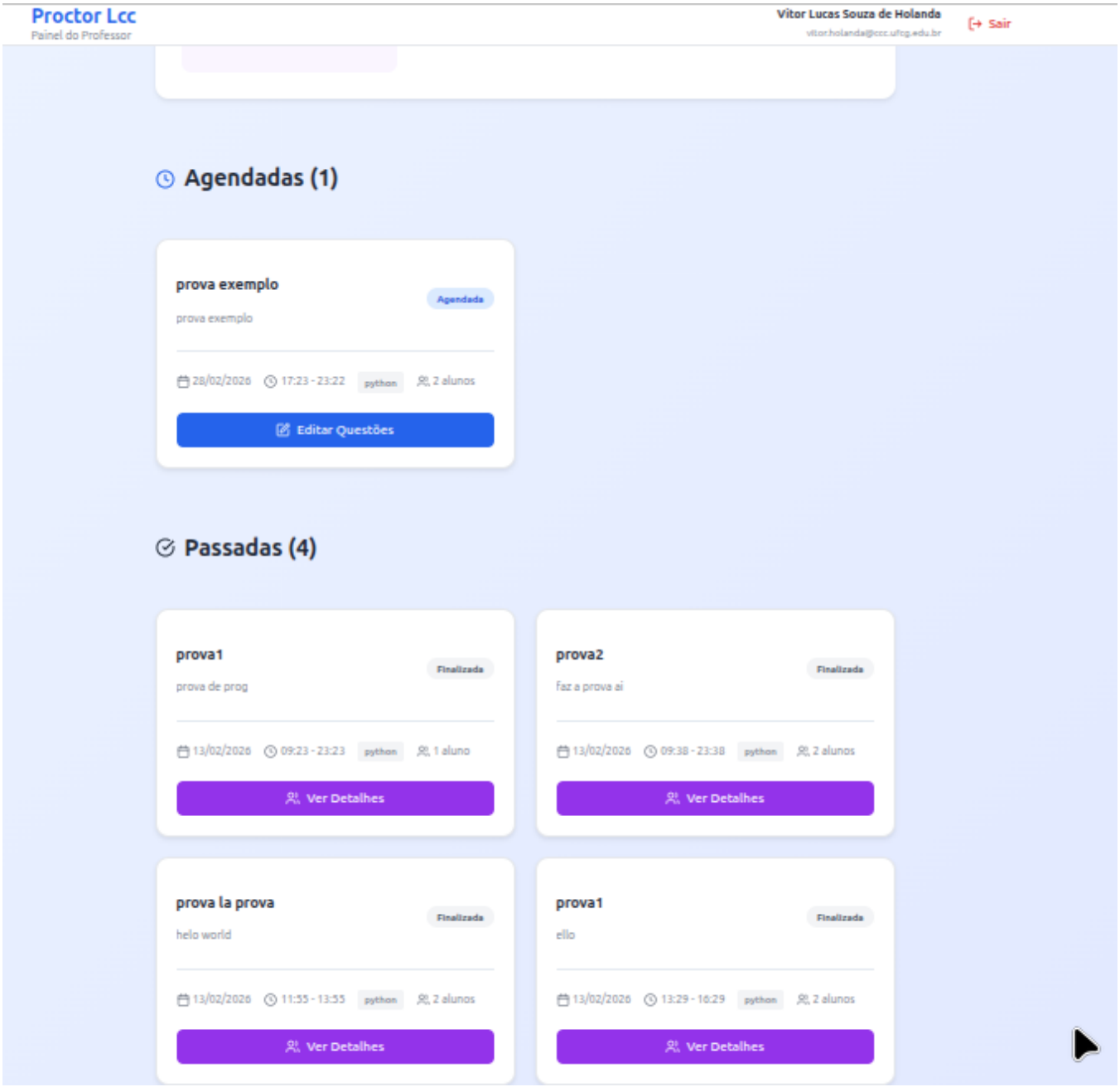


Figura 5: Painel do Professor: Detalhes de provas.

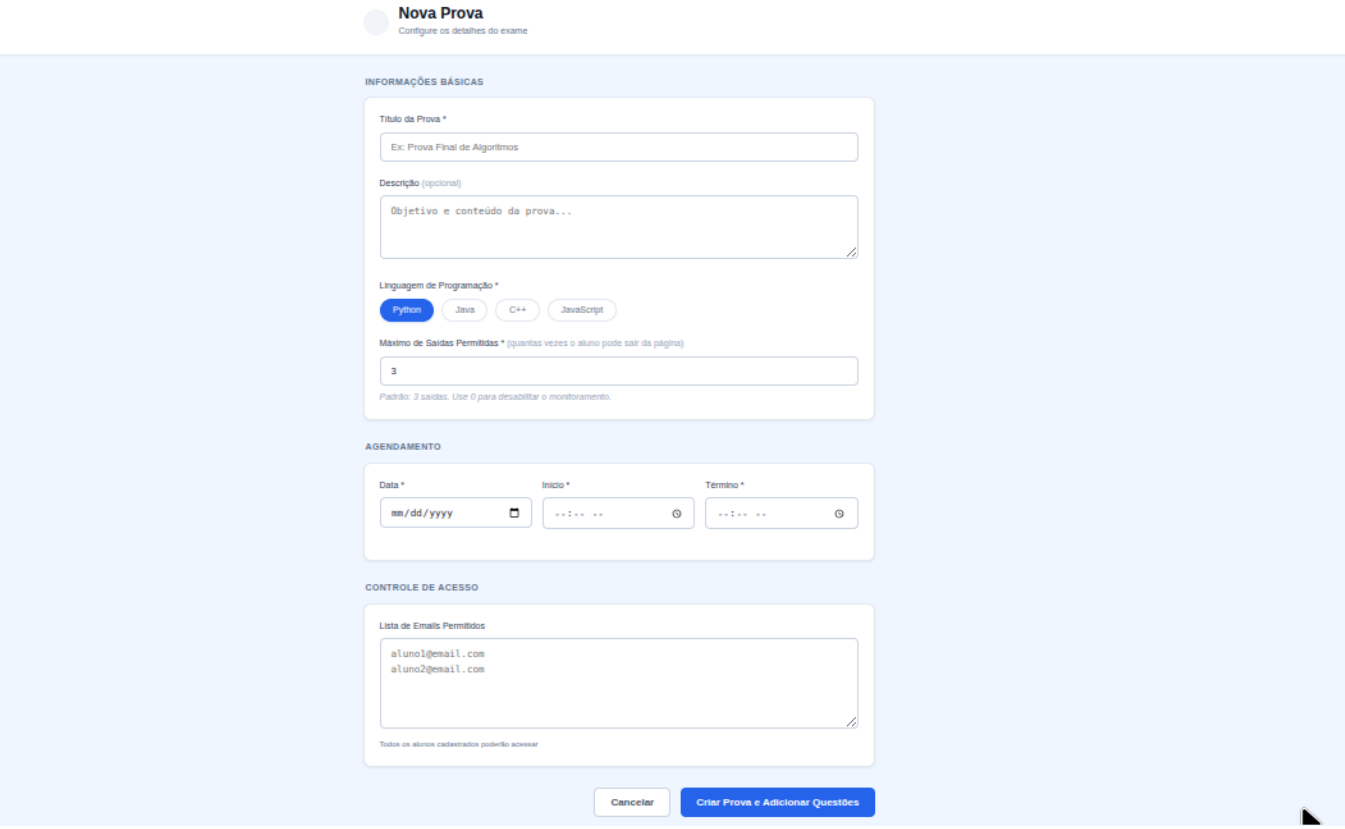


Figura 6: Interface de configuração de parâmetros e cadastro de prova.

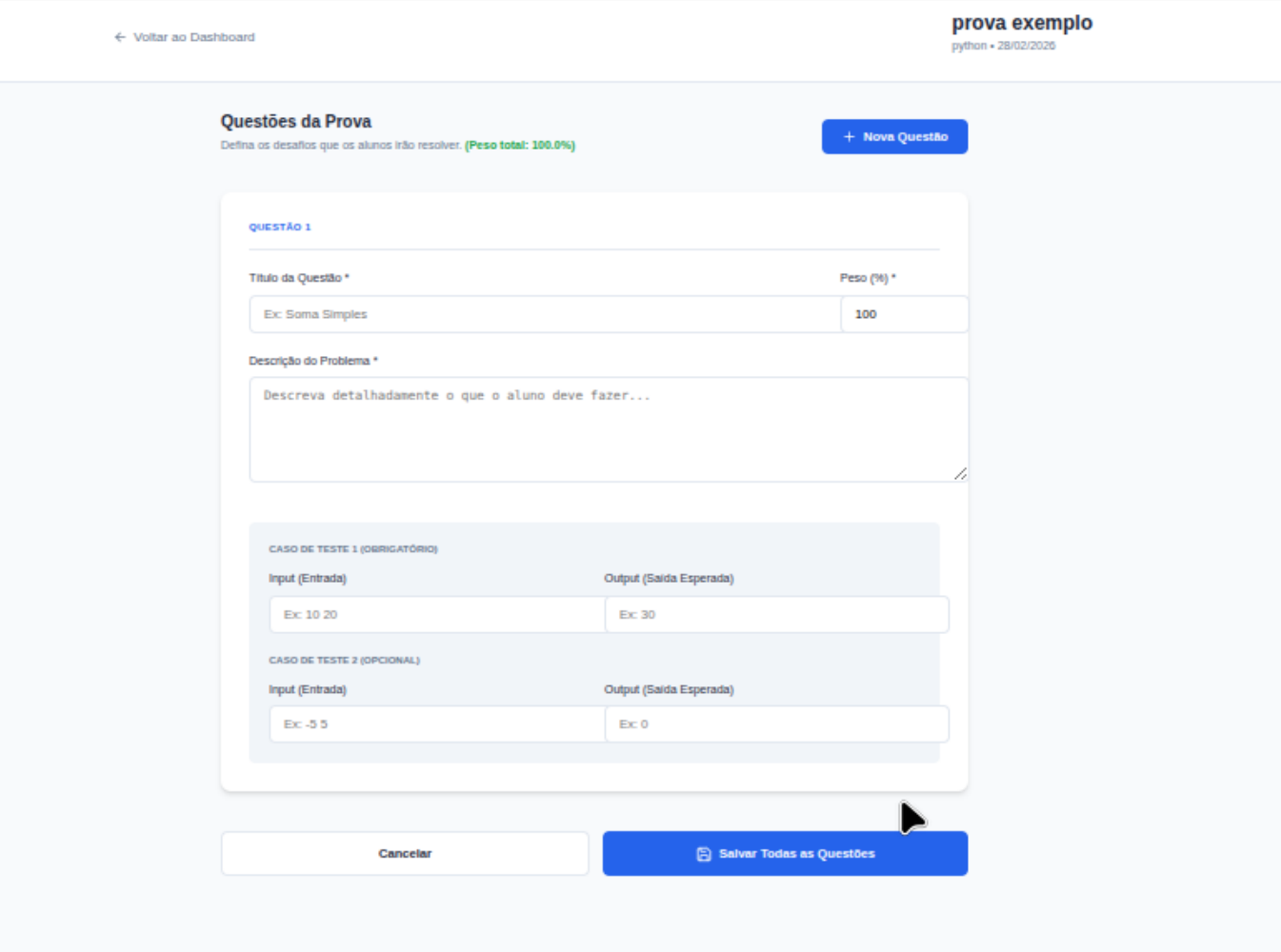


Figura 7: Interface de configuração de parâmetros e cadastro de questões.

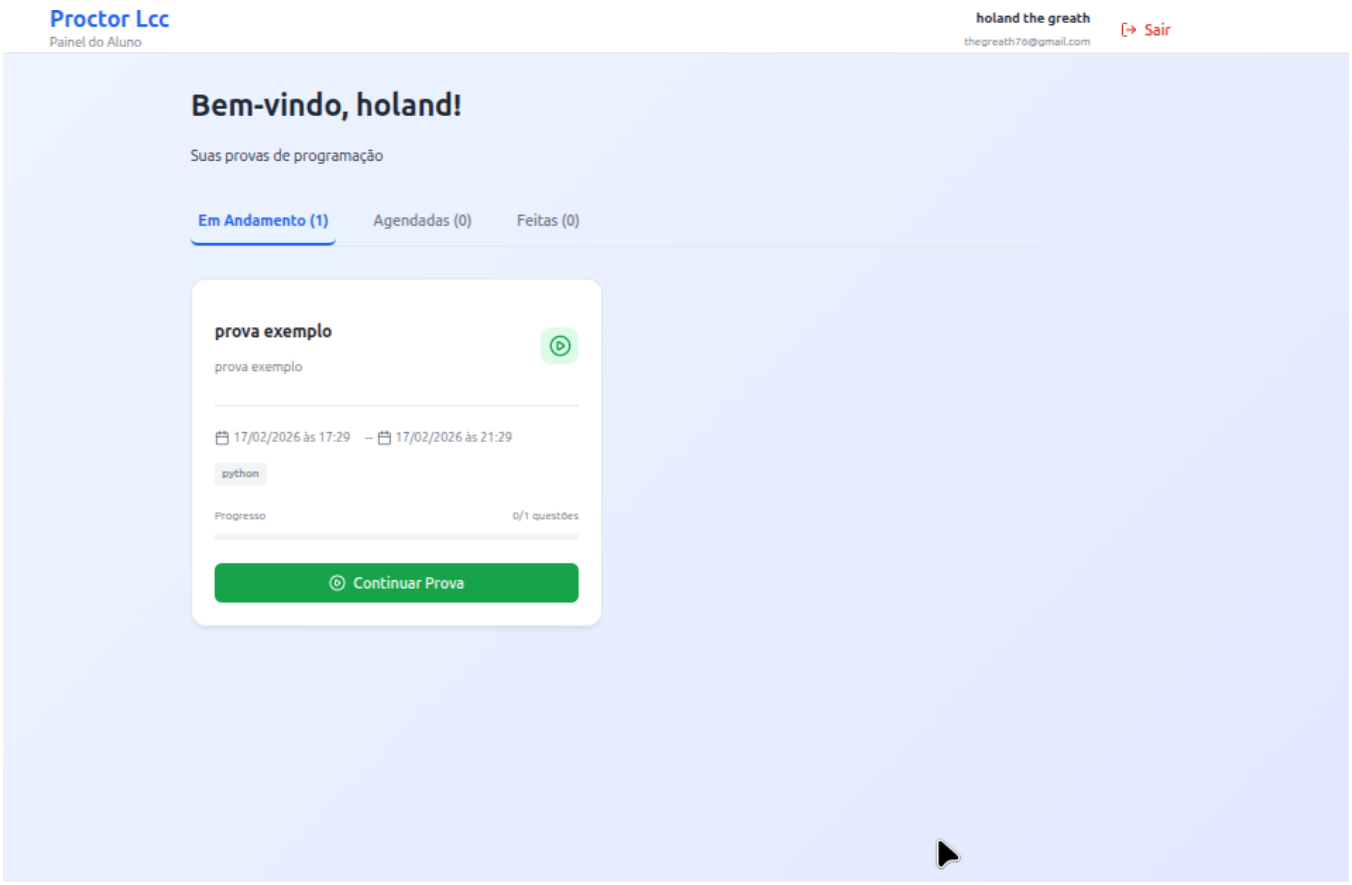


Figura 8: Painel do aluno.

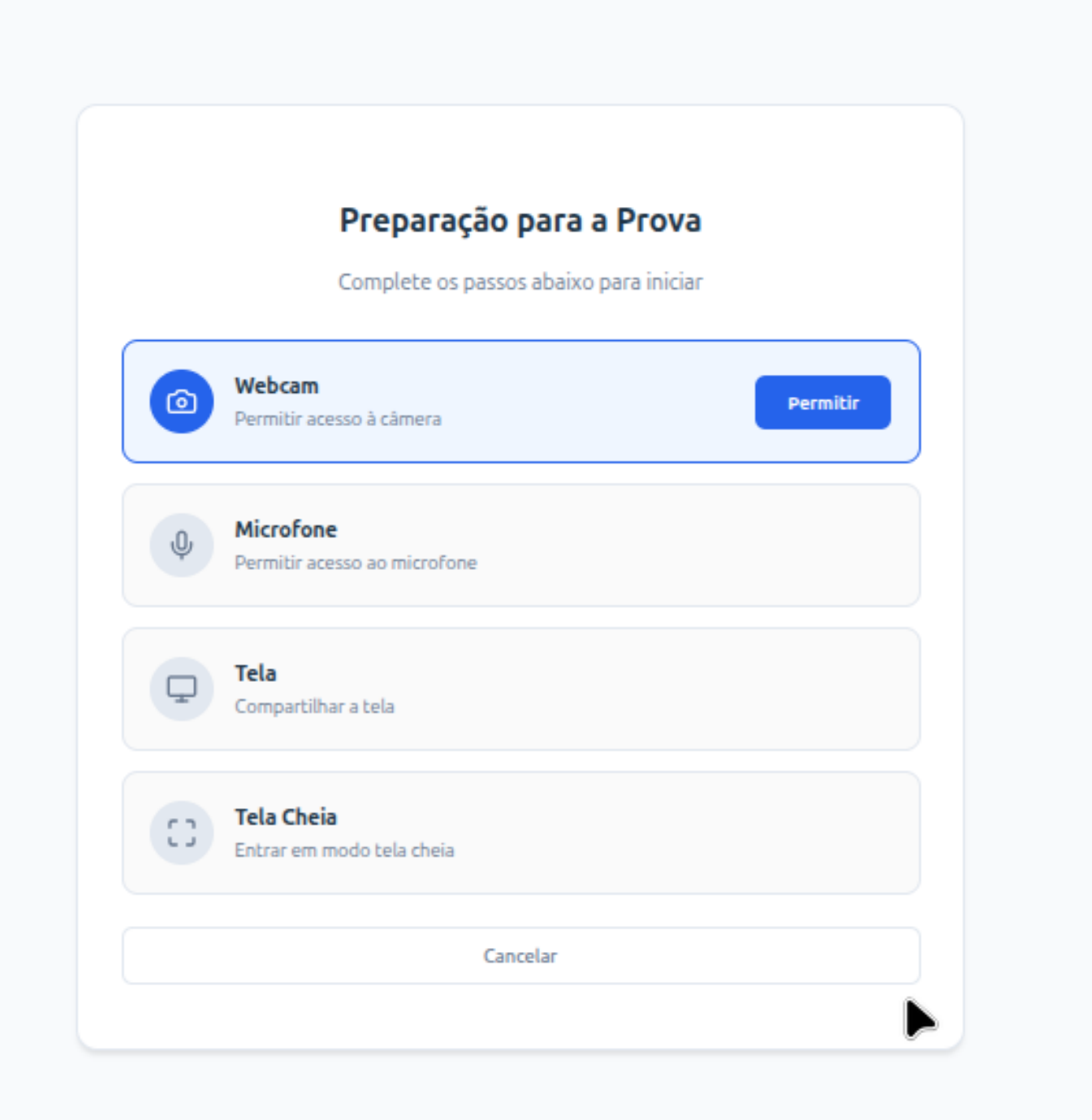


Figura 9: Interface de pré-prova: Validação de requisitos e ativação do ScreenGuard.

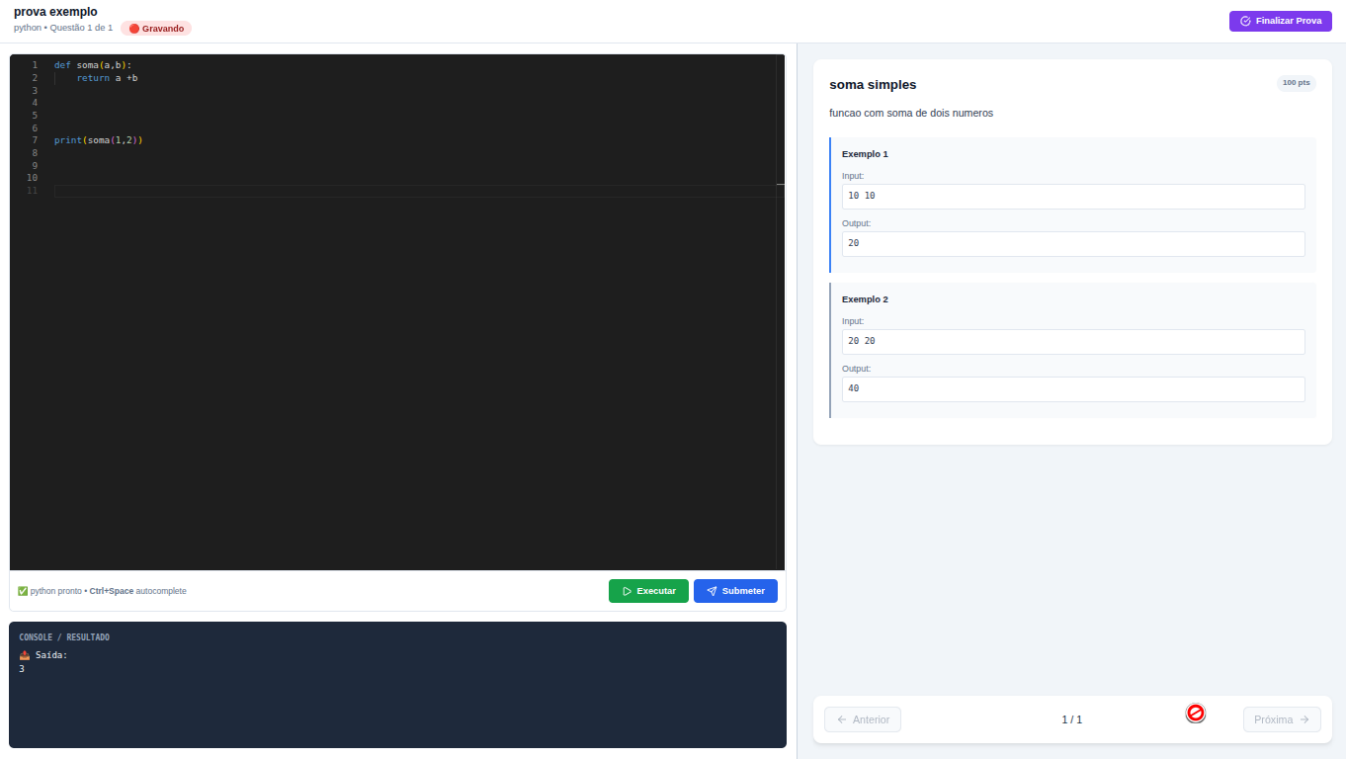


Figura 10: Interface de execução do aluno: Editor Monaco com monitoramento persistente.

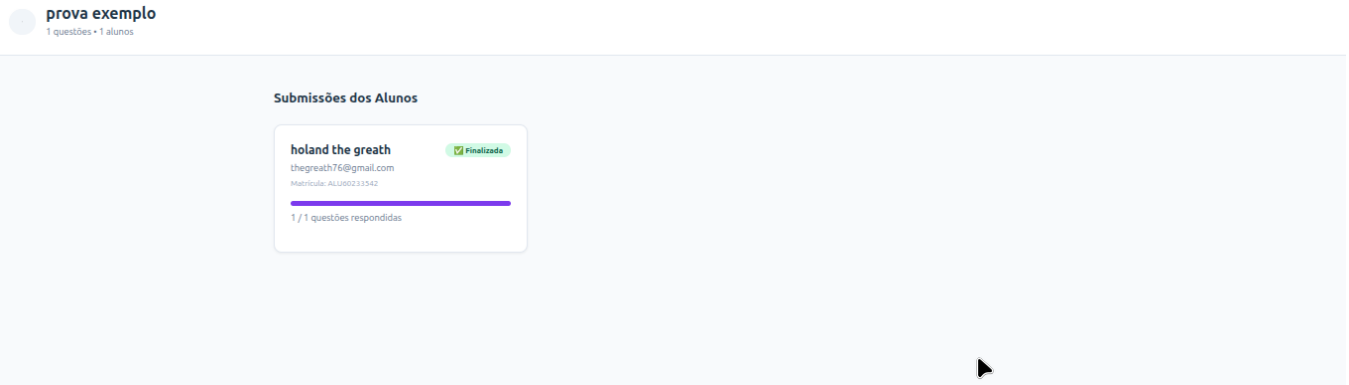


Figura 11: Página visualizacao de submissões de uma prova.

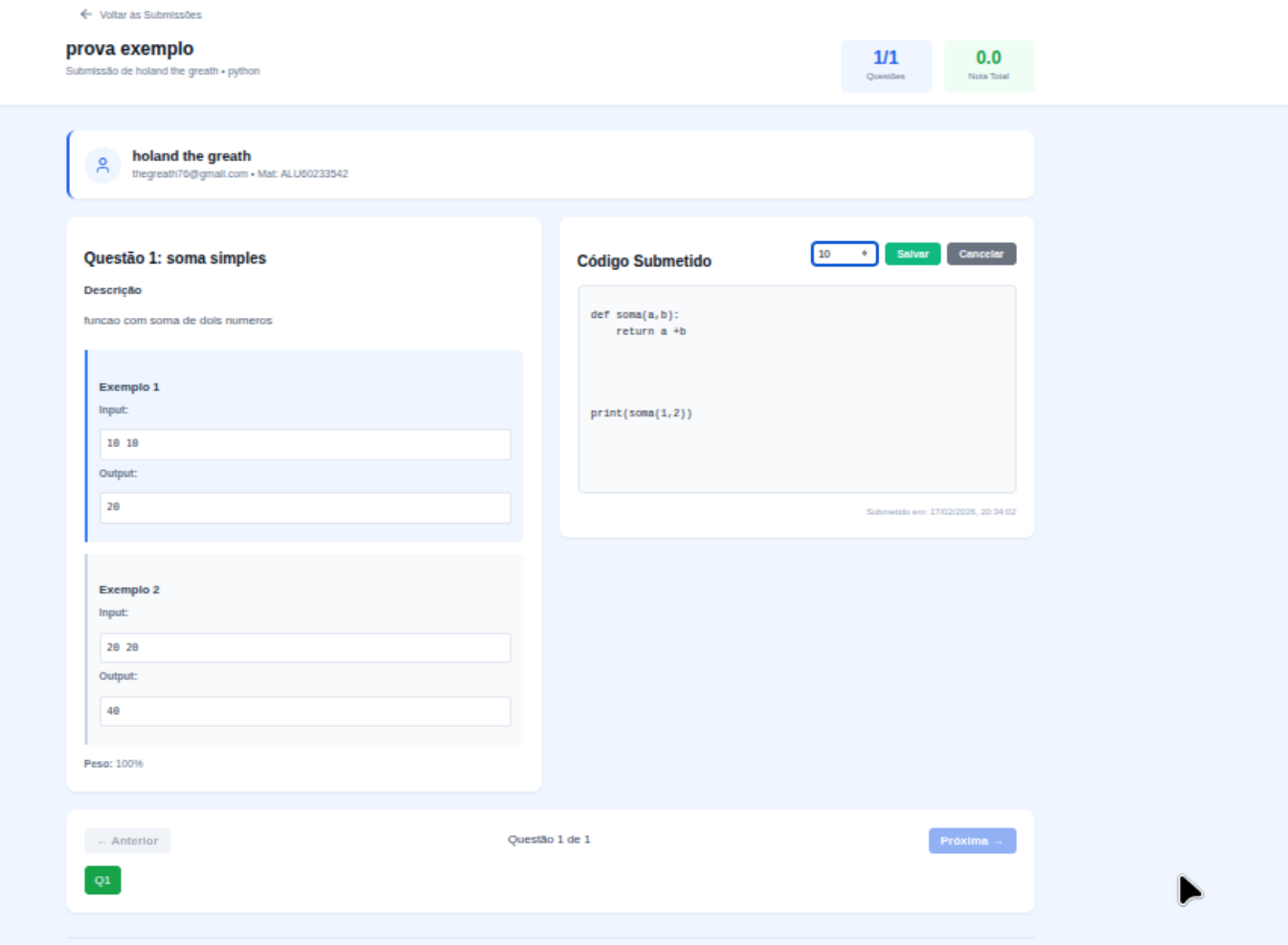


Figura 12: Painel de Auditoria: Consolidação de indícios comportamentais para correção.

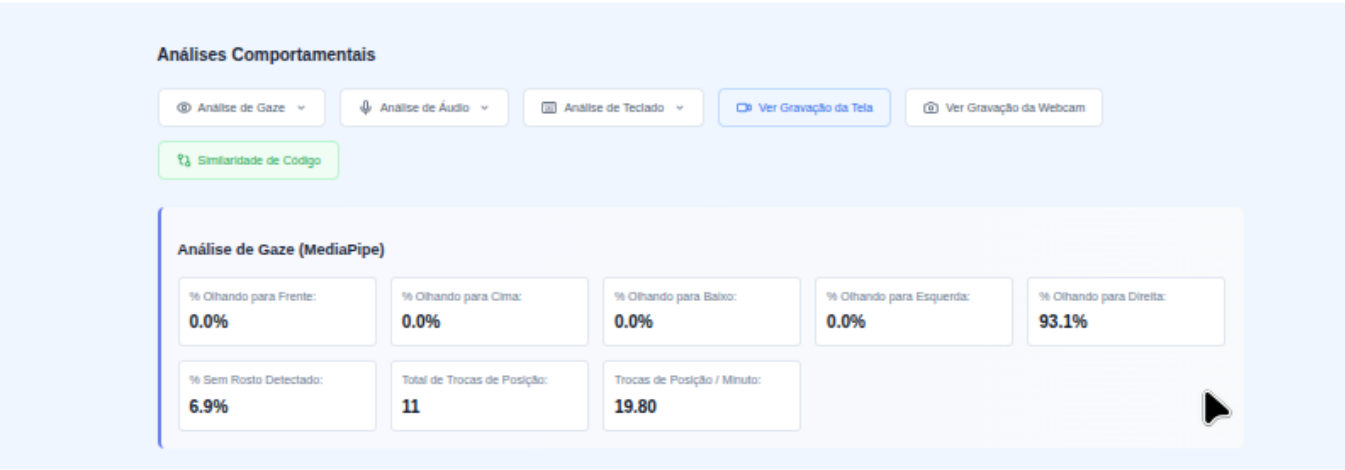


Figura 13: Resultados da análise visual e estimativa de pose da cabeça.



Figura 14: Relatório de diarização de áudio e detecção de atividades sonoras.



Figura 15: Análise da dinâmica de digitação (*keystroke*) e detecção de anomalias.

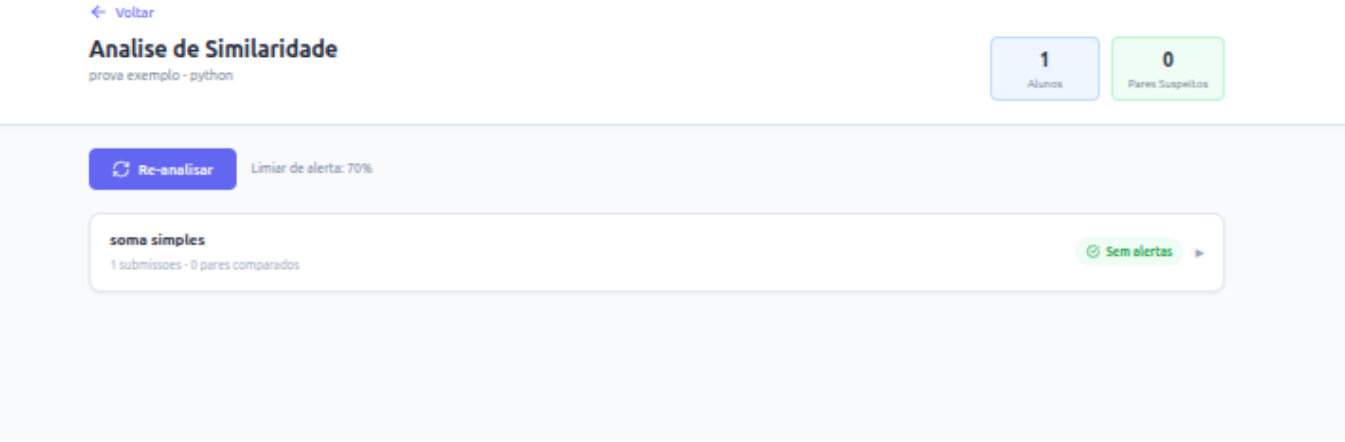


Figura 16: Painel de detecção de plágio por meio de similaridade de código.

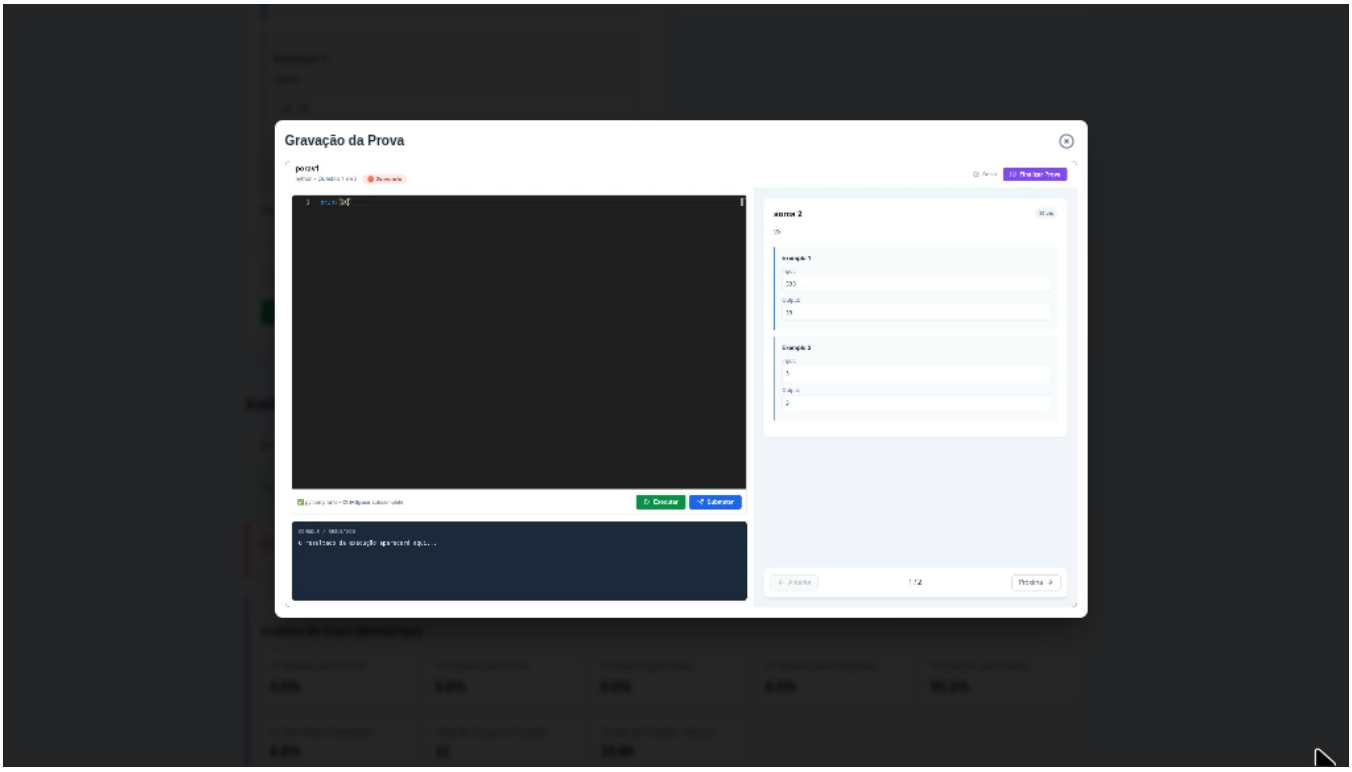


Figura 17: Reprodução das gravações de tela e webcam do aluno.

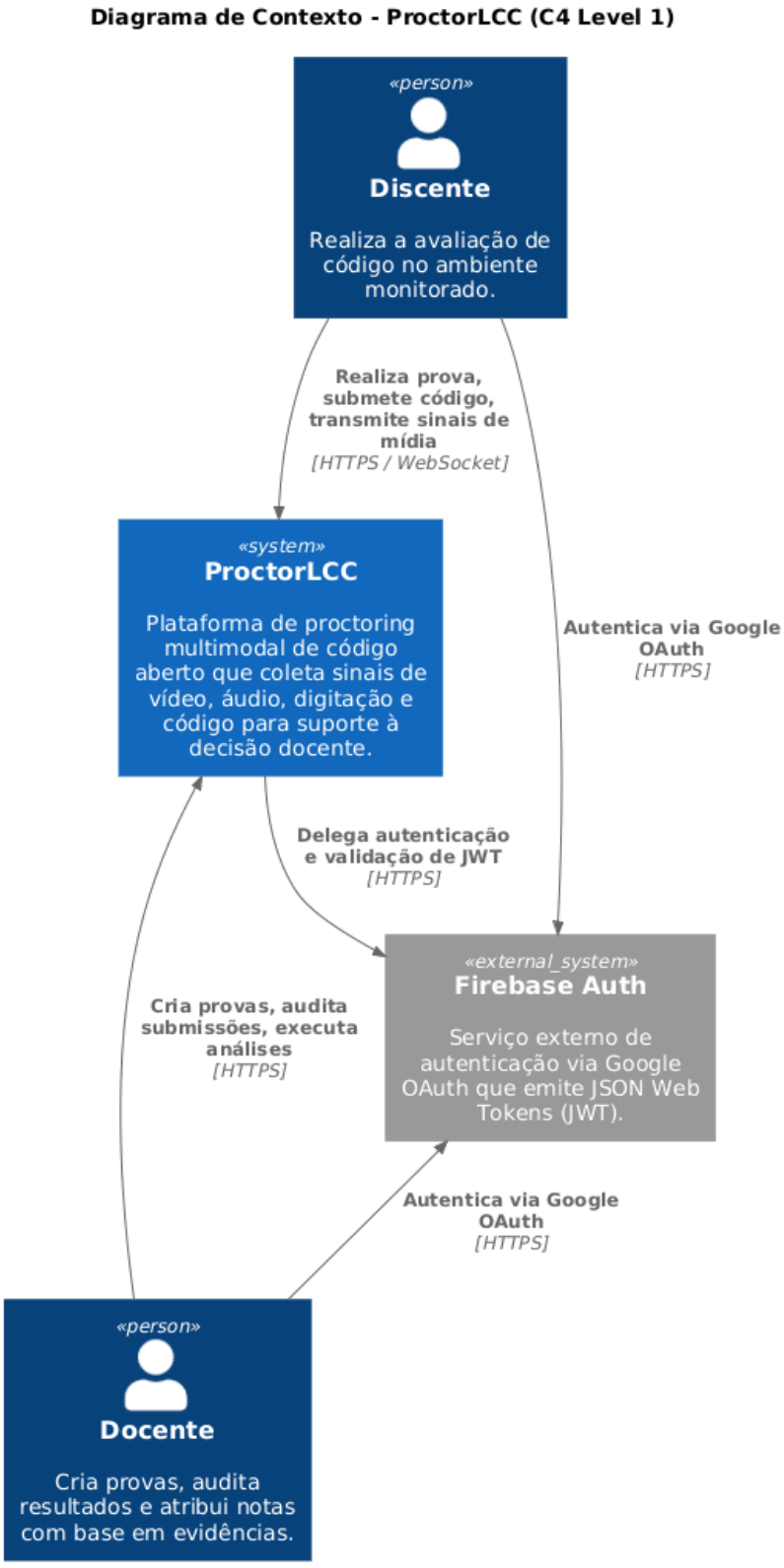


Figura 18: Diagrama de Contexto

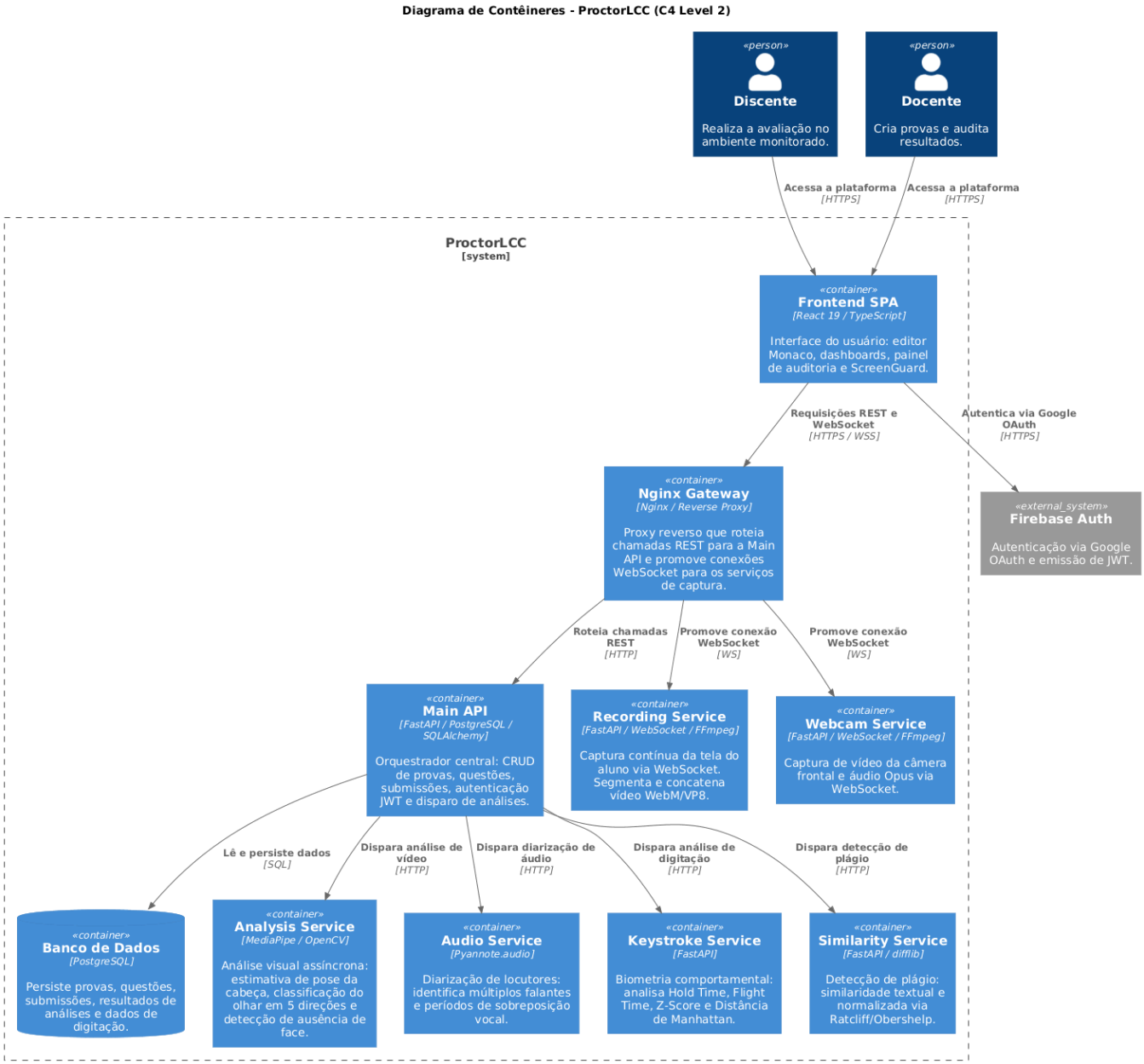
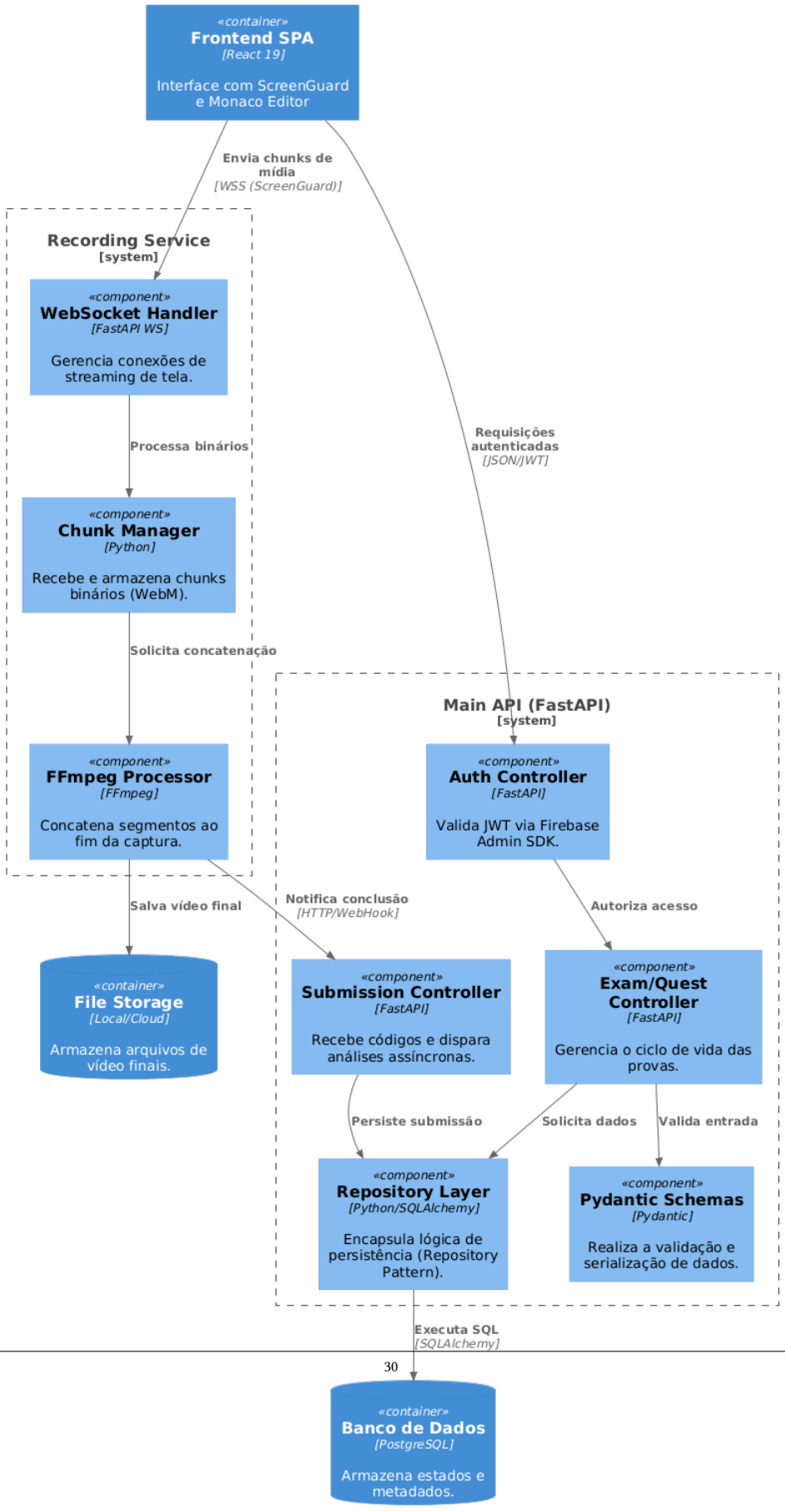


Figura 19: Diagrama de Contêineres

Diagrama de Componentes - ProctorLCC (C4 Level 3)



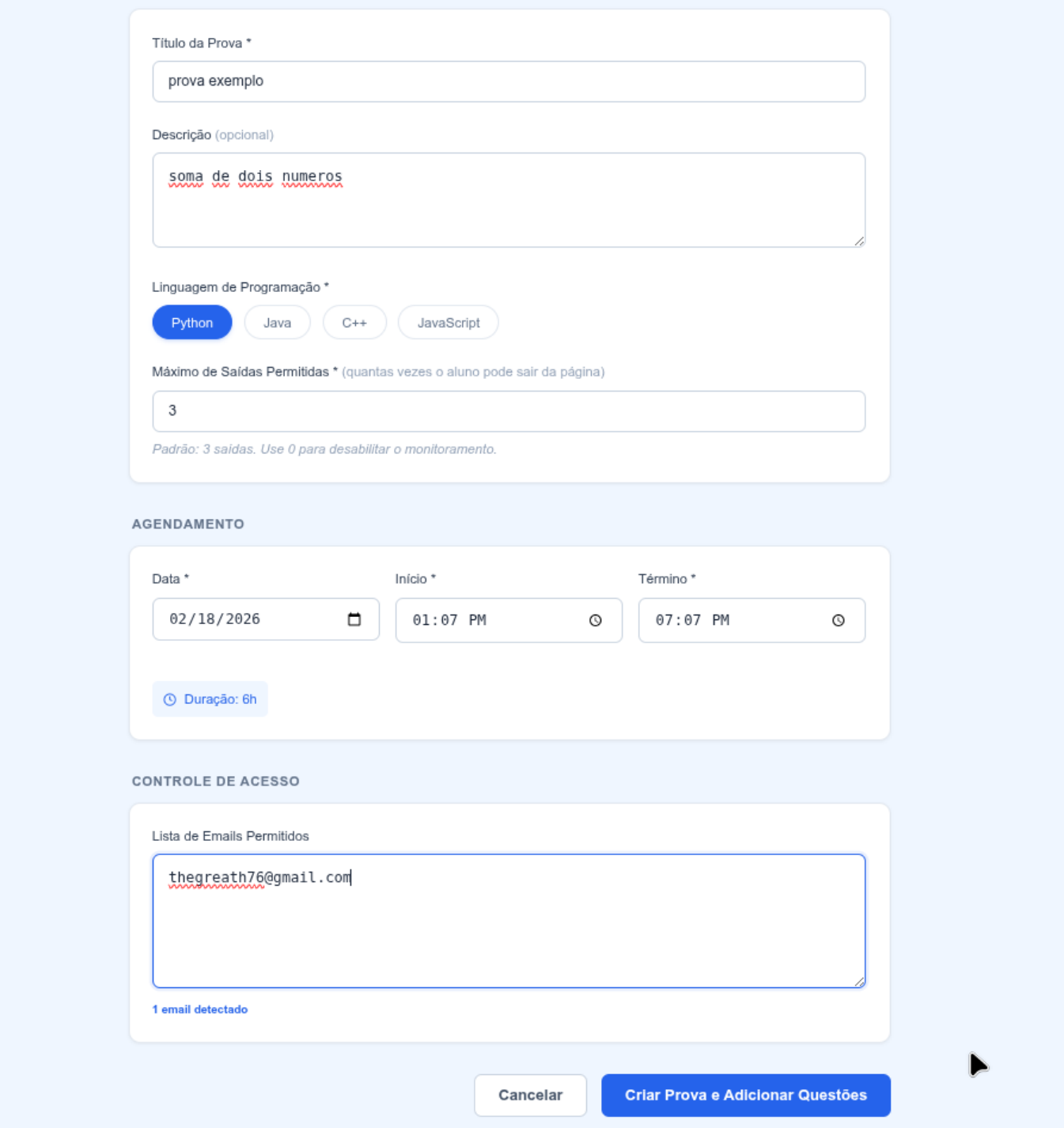


Figura 21: Interface de criação e configuração do exame pelo docente.

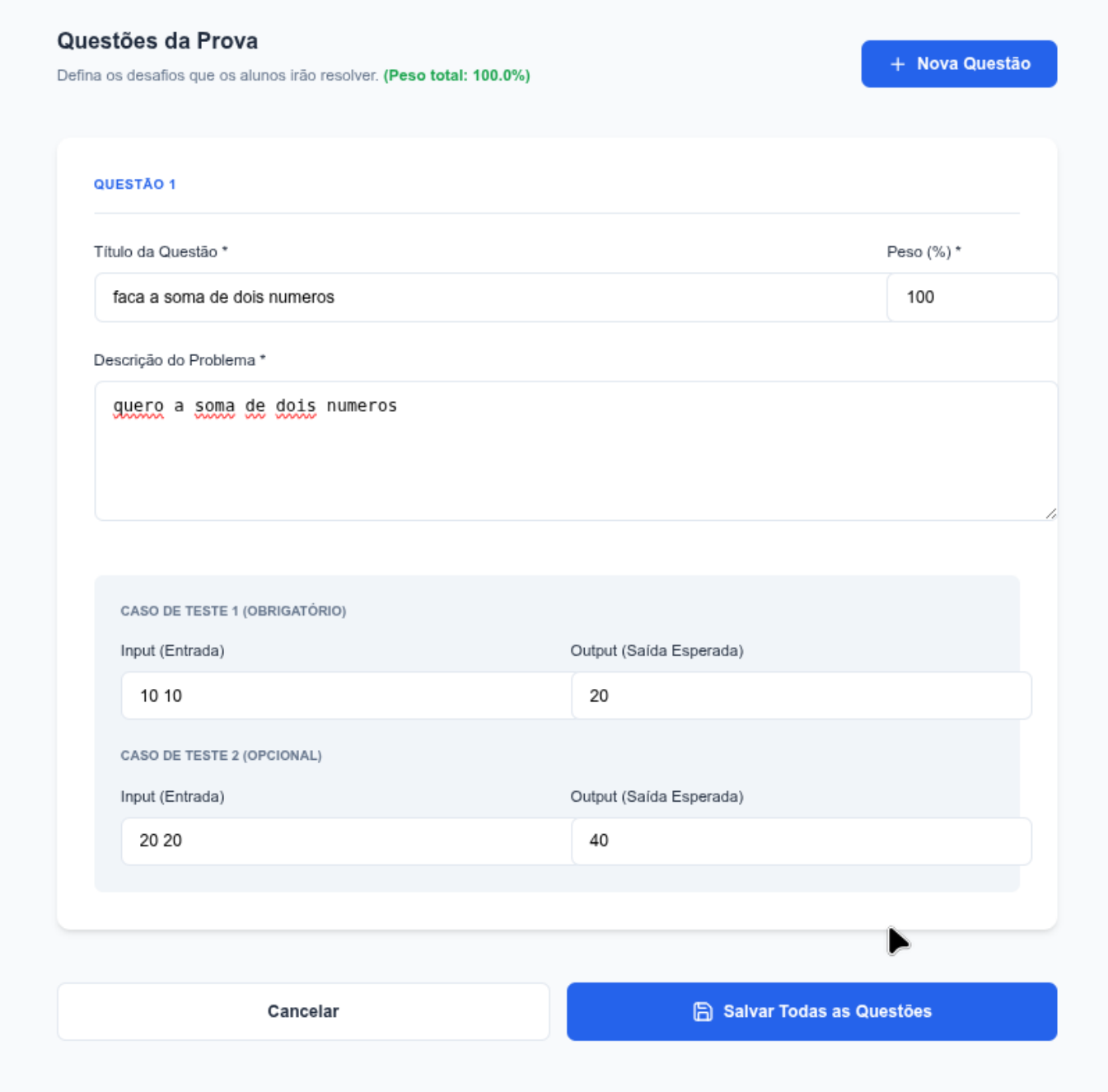


Figura 22: Configuração de questão com definição de casos de exemplo.

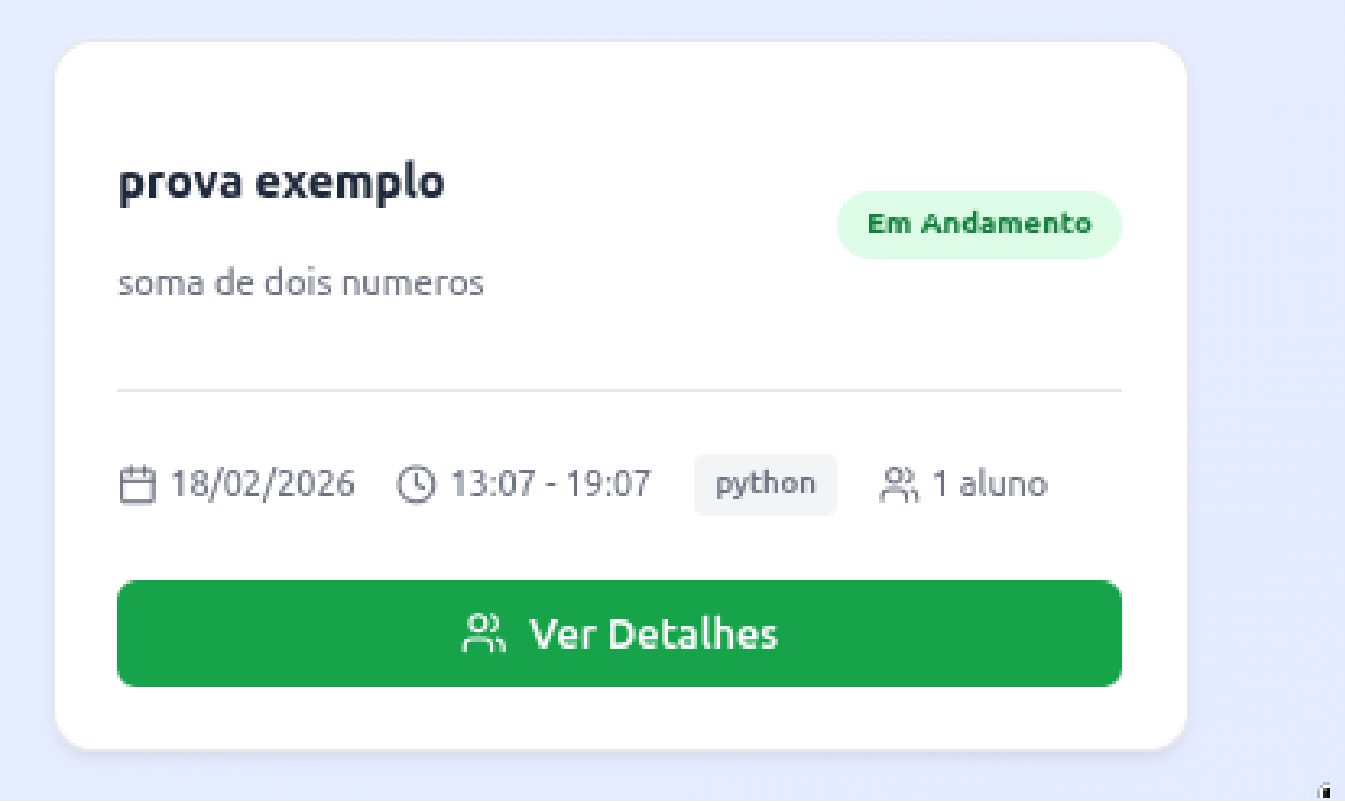


Figura 23: Visualização do monitoramento de provas ativas no painel docente.

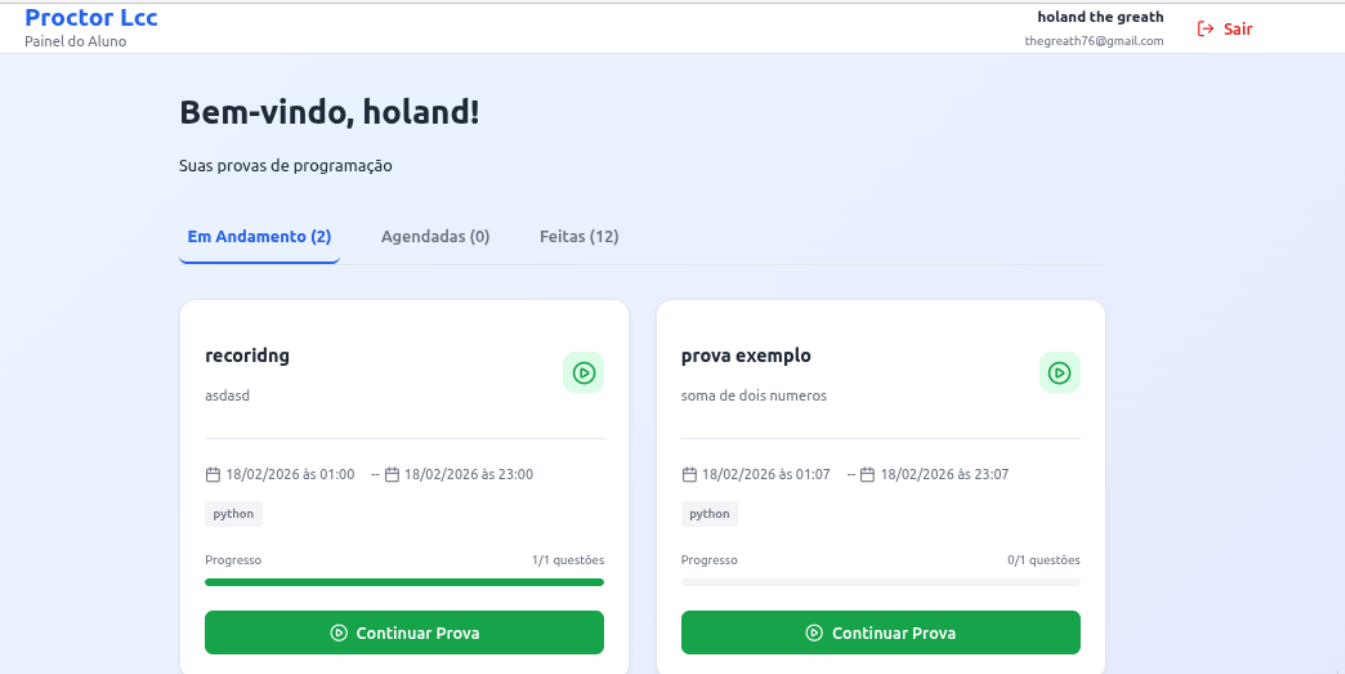


Figura 24: Painel do discente exibindo a avaliação disponível para execução.

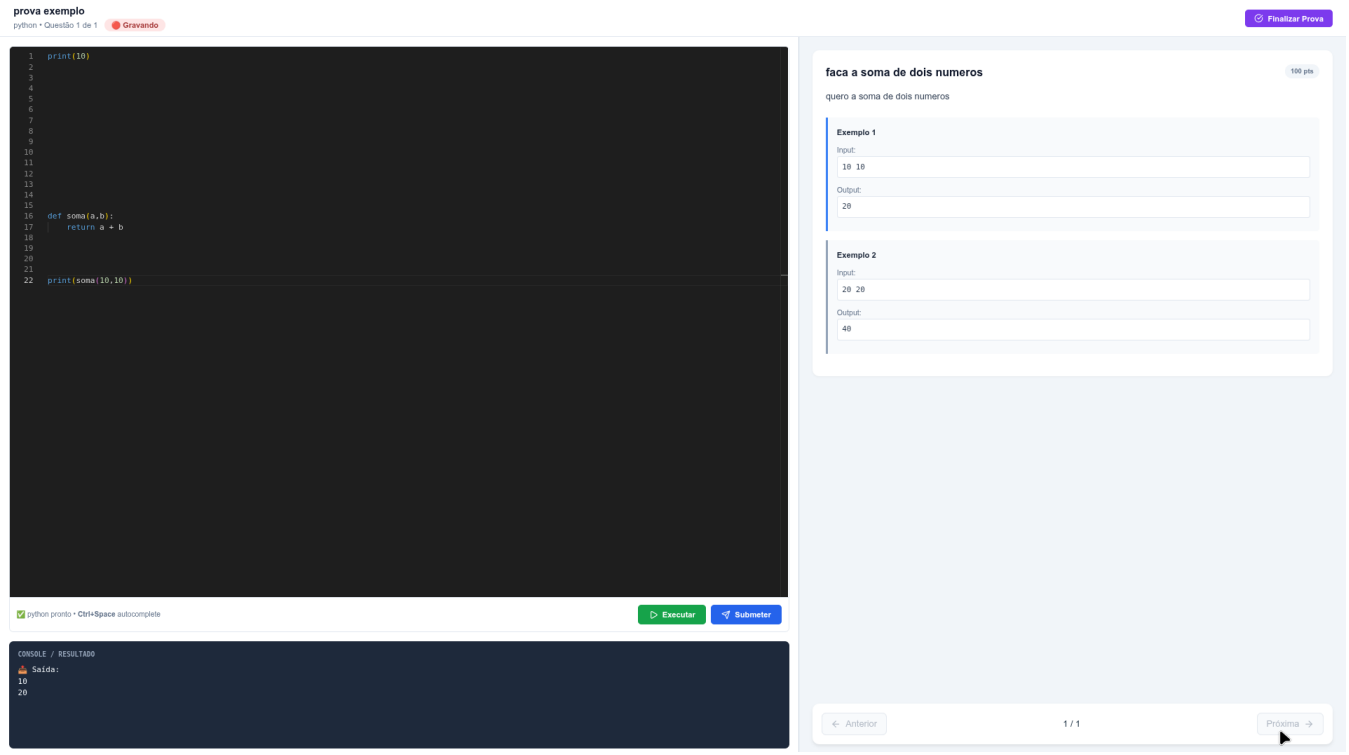


Figura 25: Ambiente de execução do exame com editor de código e monitoramento ativo.

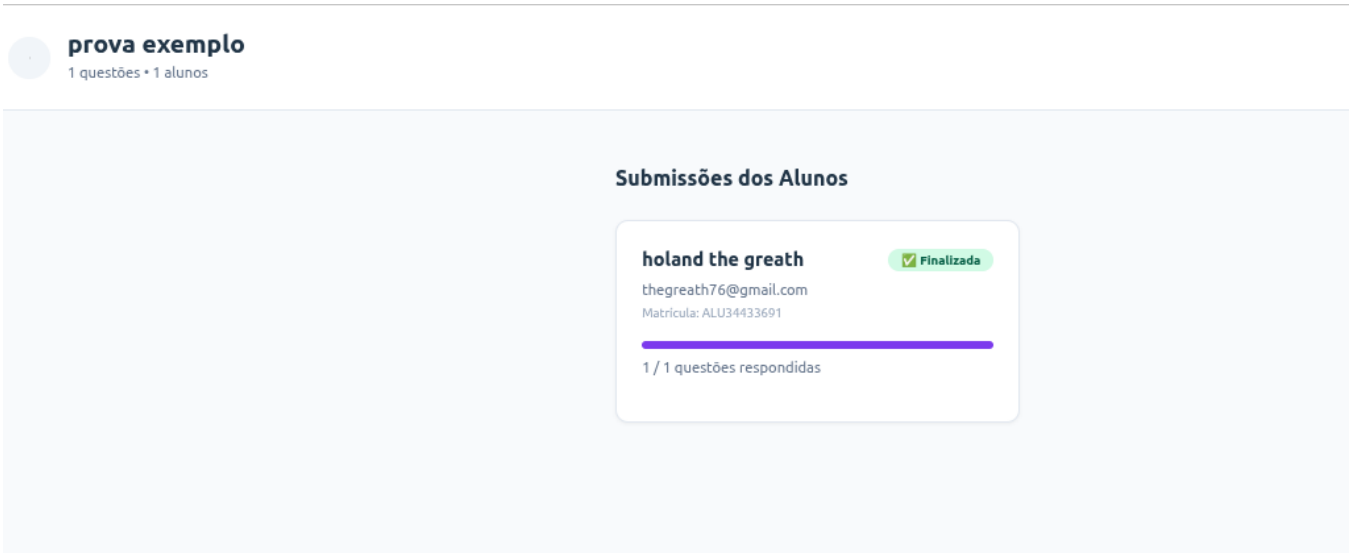


Figura 26: Listagem de submissões recebidas para o exame selecionado.

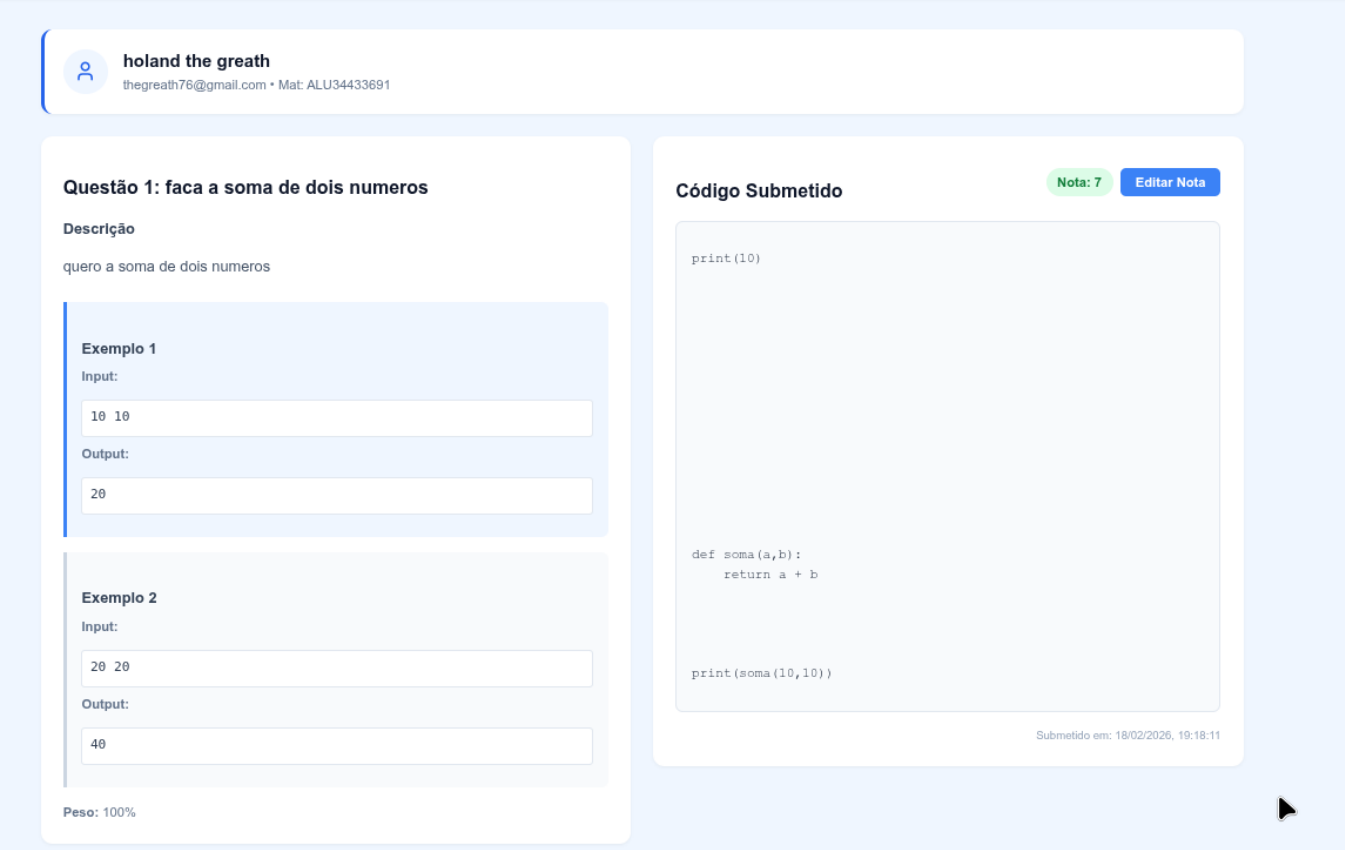


Figura 27: Visualização docente da submissão detalhada com atribuição de nota.

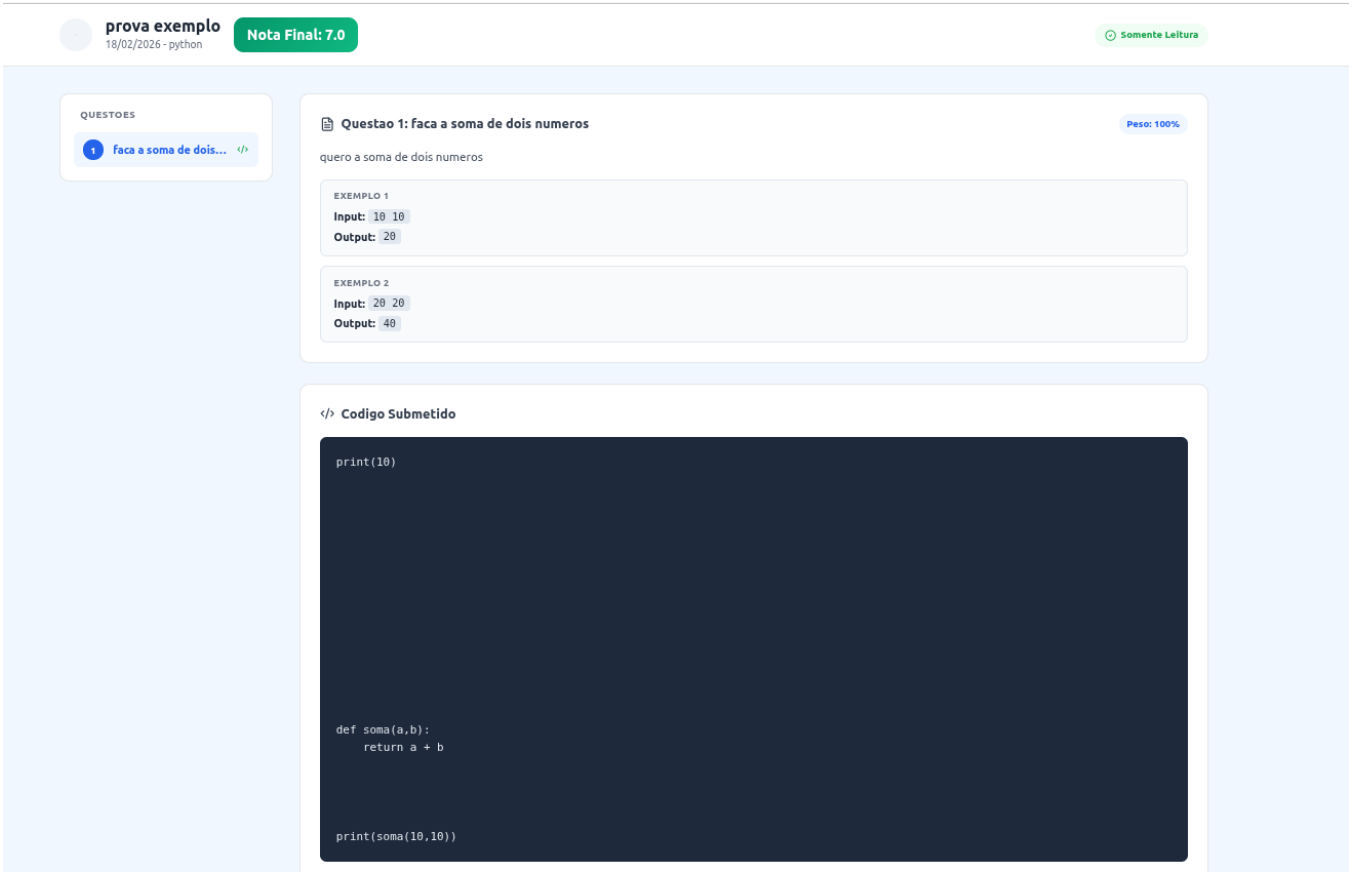


Figura 28: Interface de resultados do discente exibindo a nota e código submetido.

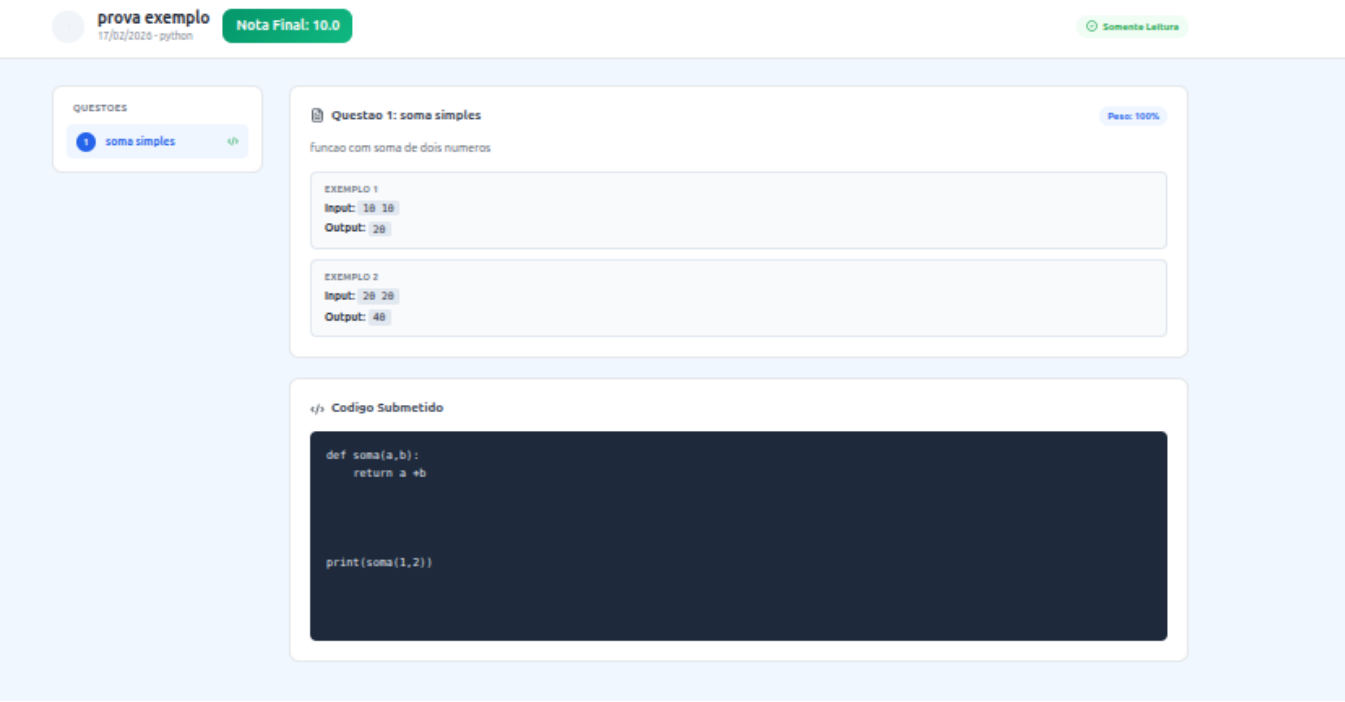


Figura 29: Página de resultados do aluno com feedback e nota final.

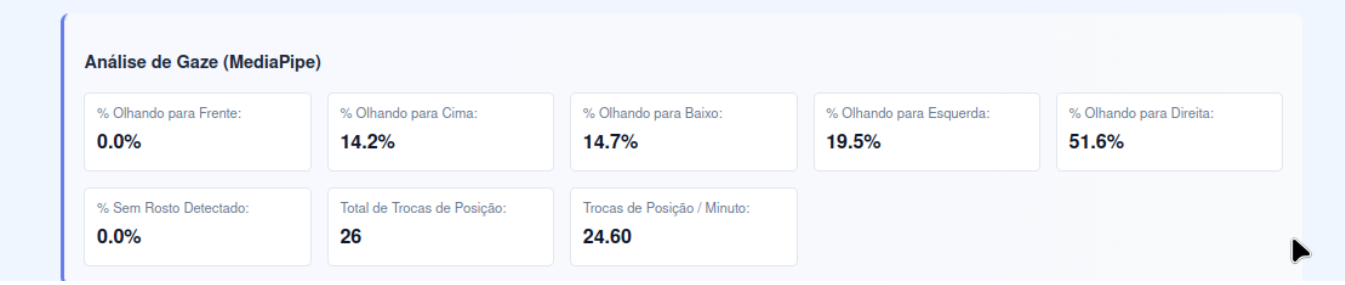


Figura 30: Relatório de *Gaze Tracking*: Percentuais de direção do olhar em teste de cinco posições.



Figura 31: Análise de robustez: Manutenção da detecção de olhar à esquerda sob rotação da câmera.

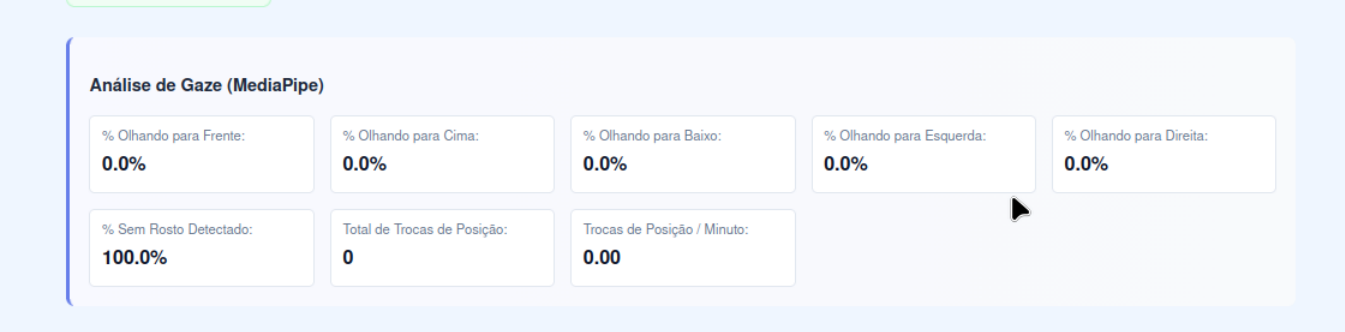


Figura 32: Detecção de irregularidade por ausência de face durante a sessão.



Figura 33: Diarização de áudio em cenário de locutor único.

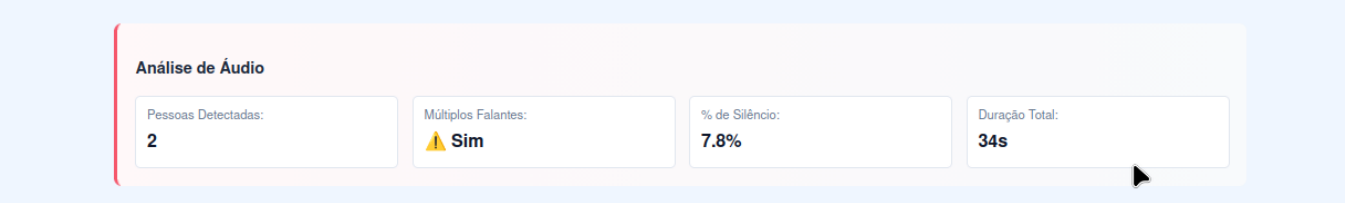


Figura 34: Diarização de áudio com múltiplos falantes e ruído de fundo.



Figura 35: Relatório de *Keystroke*: Linha de base em cenário de resolução autoral.

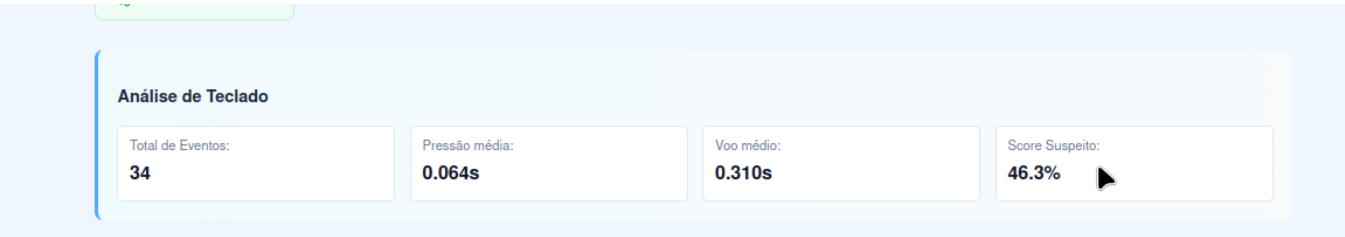


Figura 36: Relatório de *Keystroke*: Detecção de uso intensivo da área de transferência.



Figura 37: Relatório de *Keystroke*: Análise de ritmo em cenário de cópia manual.

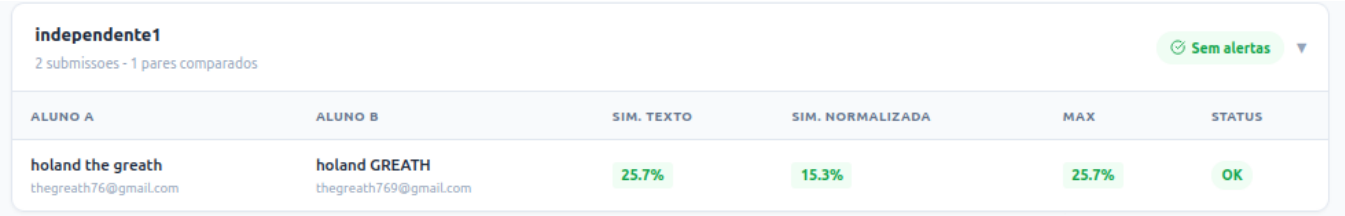


Figura 38: Resultado de similaridade para códigos independentes (Aluno 1 vs Aluno 2).

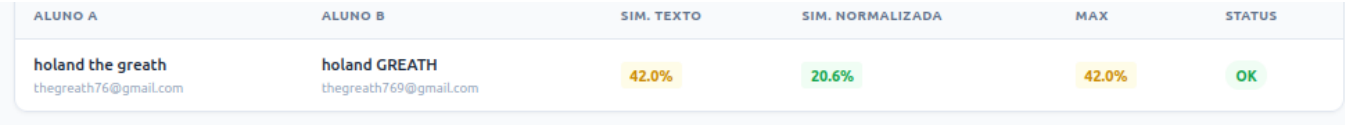


Figura 39: Resultado de similaridade para códigos com suspeita de plágio (Aluno 3 vs Aluno 4).

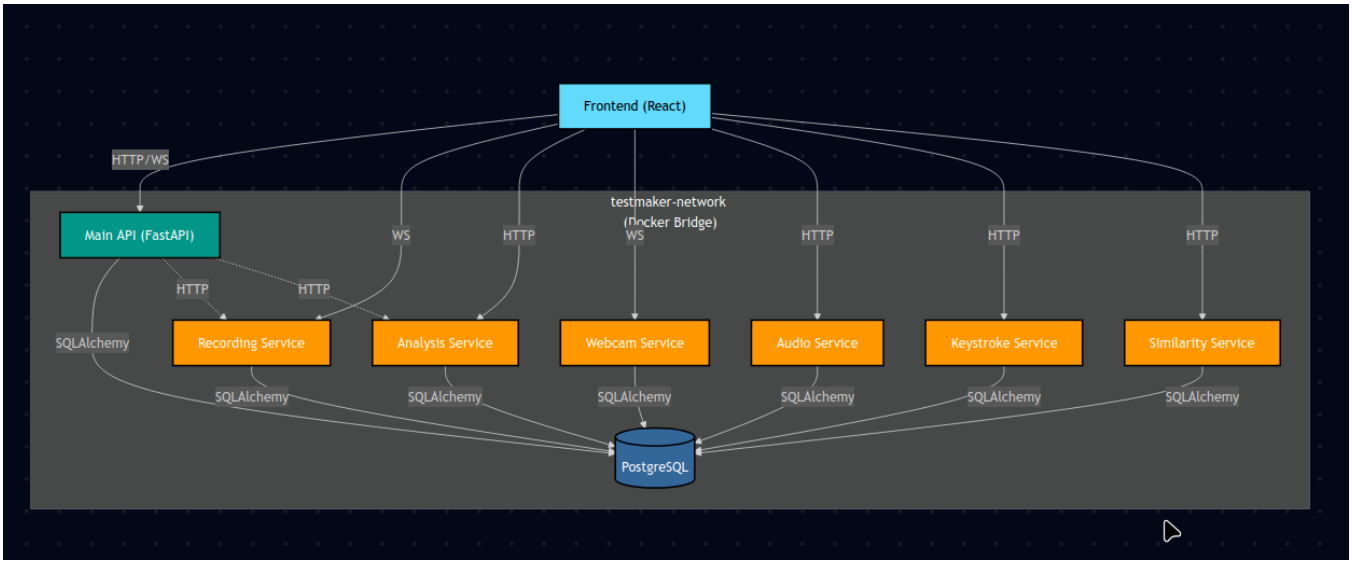


Figura 40: Arquitetura Docker do sistema.



Figura 41: Representação do funcionamento do algoritmo de estimativa de pose e projeção de eixos.